

## Course 3: Unsupervised Learning



**IMT Atlantique**  
Bretagne-Pays de la Loire  
École Mines-Télécom

2026-08-31

Course 3: Unsupervised Learning

Course 3: Unsupervised Learning



## Summary

| Last session                                                                                                                                                                                | Today's session                                                                                                                                                                                                                                              |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>Supervised learning - learning from labeled examples</li><li>Challenges of supervised learning</li><li>Evaluation: ground truth and metrics</li></ul> | <ul style="list-style-type: none"><li>Learning from unlabeled examples<ul style="list-style-type: none"><li>Clustering</li><li>Decomposition</li><li>Manifold learning</li></ul></li><li>Evaluating without labels</li><li>A note on data cleaning</li></ul> |

### Last session

- 1 Supervised learning - learning from labeled examples
- 2 Challenges of supervised learning
- 3 Evaluation: ground truth and metrics

### Today's session

- 1 Learning from unlabeled examples
  - 1 Clustering
  - 2 Decomposition
  - 3 Manifold learning
- 2 Evaluating without labels
- 3 A note on data cleaning

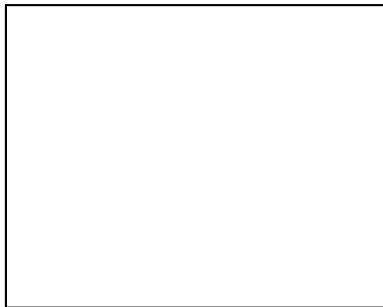
Last session was about *supervised* learning: pairs (input, label), and the goal of predicting the label of inputs never seen. Today the labels disappear.

The question is no longer "what is the right answer for this point?" but "how is this data organised?". Three families of methods answer it, followed by two transversal topics: how to evaluate when there is no ground truth, and how to prepare your features.



## Notations

Vector space ( $\mathbb{R}^d$ )



Same notations as last session, with one difference: there is no  $y$  anymore.

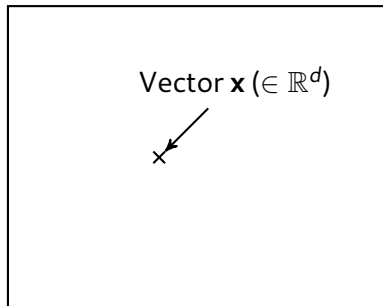
A dataset is a set  $X$  of  $N$  vectors in dimension  $d$ . Everything we do today uses only the geometry of this cloud: which points are close to which, in which directions the cloud is stretched, whether it lies on a curve.

Keep in mind that “close” depends on a distance, and that this distance depends on the units of your features. This is exactly why the last section of the session is devoted to preparing them.



## Notations

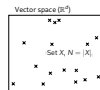
Vector space ( $\mathbb{R}^d$ )



Same notations as last session, with one difference: there is no  $y$  anymore.

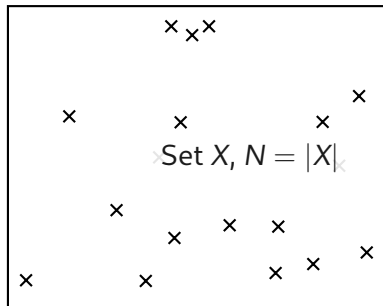
A dataset is a set  $X$  of  $N$  vectors in dimension  $d$ . Everything we do today uses only the geometry of this cloud: which points are close to which, in which directions the cloud is stretched, whether it lies on a curve.

Keep in mind that "close" depends on a distance, and that this distance depends on the units of your features. This is exactly why the last section of the session is devoted to preparing them.



## Notations

Vector space ( $\mathbb{R}^d$ )

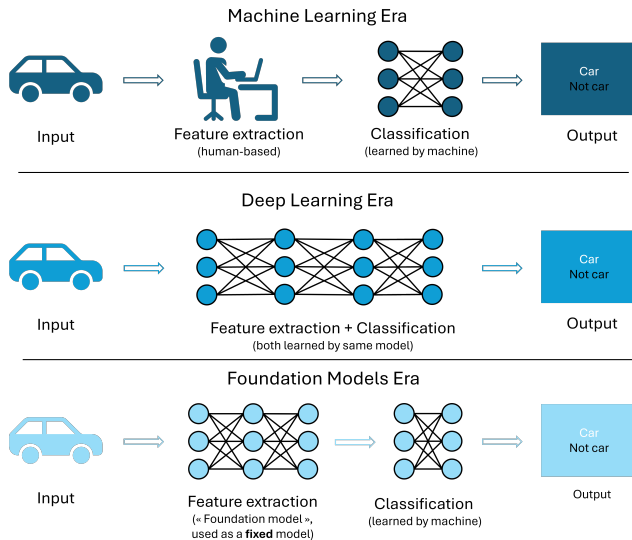


Same notations as last session, with one difference: there is no  $y$  anymore.

A dataset is a set  $X$  of  $N$  vectors in dimension  $d$ . Everything we do today uses only the geometry of this cloud: which points are close to which, in which directions the cloud is stretched, whether it lies on a curve.

Keep in mind that "close" depends on a distance, and that this distance depends on the units of your features. This is exactly why the last section of the session is devoted to preparing them.

# What is the vector $x$ ?

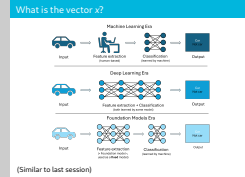


(Similar to last session)

2026-08-31

## Course 3: Unsupervised Learning

What is the vector  $x$ ?



In traditional Machine Learning, data is generally a feature representation of the data (i.e. the edges or the colorimetry levels in an image, or spectrograms in audio).

A first shift appears with Deep Learning, where data is fed as it is to models, allowing to alleviate the complex, time-consuming and hand-crafted feature engineering, and leading to impressive performance in image processing, the drawback being a need for large amounts of data and computing power.

Finally, a recent shift appears with larger and more sophisticated models ("Foundation Models"), where they can be used as a first pre-processing step. The resulting representations are called "embeddings", and are vectors in the latent space of the model, i.e. the input vectors (image, text, ...) that have been mapped in a lower dimensional space by a function  $f : R^d \rightarrow R^l$ . Embeddings are usually richer semantically and easier to manipulate for a downstream task (e.g. classification).

# Unsupervised learning

## Goal

Discover patterns/structure in  $X$ ,

## Unsupervised learning

- Unsupervised = no expert, no labels
- Main approaches:
  - Clustering = find a partition of  $X$  in  $K$  subsets,
  - Decomposition using  $K$  vectors,
  - Manifold Learning.
- Applications :
  - Dimensionality reduction,
  - Quantization
  - Visualization...



2026-08-31

## Course 3: Unsupervised Learning

### └ Unsupervised learning

Unsupervised learning

Goal  
Discover patterns/structure in  $X$ ,

Unsupervised learning

- Unsupervised = no expert, no labels
- Main approaches:
  - Clustering = find a partition of  $X$  in  $K$  subsets,
  - Decomposition using  $K$  vectors,
  - Manifold Learning.

Applications :

- Dimensionality reduction,
- Quantization
- Visualization...

Three ways of saying "there is structure in this cloud".

Clustering answers *which points go together*: the answer is one integer per point. Decomposition answers *along which directions the cloud lives*: a few coordinates per point, in a basis estimated from the data. Manifold learning answers *what is the intrinsic shape of the cloud*: here the points are 2D but all lie close to a curve, so a single curvilinear coordinate almost describes them.

All three reduce the amount of information needed to describe a point. This is why the same two applications keep coming back: compression and visualization.

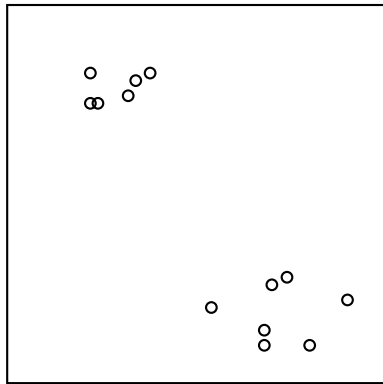
# Unsupervised learning

## Goal

Discover patterns/structure in  $X$ ,

## Unsupervised learning

- Unsupervised = no expert, no labels
- Main approaches:
  - Clustering = find a partition of  $X$  in  $K$  subsets,
  - Decomposition using  $K$  vectors.
  - Manifold Learning.
- Applications :
  - Dimensionality reduction, Quantization
  - Visualization...



2026-08-31

## Course 3: Unsupervised Learning

### └ Unsupervised learning

Unsupervised learning

Goal:  
Discover patterns/structure in  $X$ .

Unsupervised learning

- Unsupervised = no expert, no labels
- Main approaches:
  - Clustering = find a partition of  $X$  in  $K$  subsets,
  - Decomposition using  $K$  vectors.
  - Manifold Learning.
- Applications :
  - Dimensionality reduction, Quantization
  - Visualization...

Three ways of saying "there is structure in this cloud".

Clustering answers *which points go together*: the answer is one integer per point. Decomposition answers *along which directions the cloud lives*: a few coordinates per point, in a basis estimated from the data. Manifold learning answers *what is the intrinsic shape of the cloud*: here the points are 2D but all lie close to a curve, so a single curvilinear coordinate almost describes them.

All three reduce the amount of information needed to describe a point. This is why the same two applications keep coming back: compression and visualization.



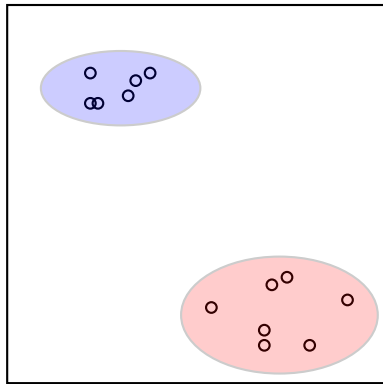
# Unsupervised learning

## Goal

Discover patterns/structure in  $X$ ,

## Unsupervised learning

- Unsupervised = no expert, no labels
- Main approaches:
  - Clustering = find a partition of  $X$  in  $K$  subsets,
  - Decomposition using  $K$  vectors.
  - Manifold Learning.
- Applications :
  - Dimensionality reduction, Quantization
  - Visualization...



2026-08-31

## Course 3: Unsupervised Learning

### └ Unsupervised learning

Unsupervised learning

Goal:  
Discover patterns/structure in  $X$ .

Unsupervised learning

- Unsupervised = no expert, no labels
- Main approaches:
  - Clustering = find a partition of  $X$  in  $K$  subsets,
  - Decomposition using  $K$  vectors.
  - Manifold Learning.
- Applications :
  - Dimensionality reduction, Quantization
  - Visualization...

Three ways of saying "there is structure in this cloud".

Clustering answers *which points go together*: the answer is one integer per point. Decomposition answers *along which directions the cloud lives*: a few coordinates per point, in a basis estimated from the data. Manifold learning answers *what is the intrinsic shape of the cloud*: here the points are 2D but all lie close to a curve, so a single curvilinear coordinate almost describes them.

All three reduce the amount of information needed to describe a point. This is why the same two applications keep coming back: compression and visualization.

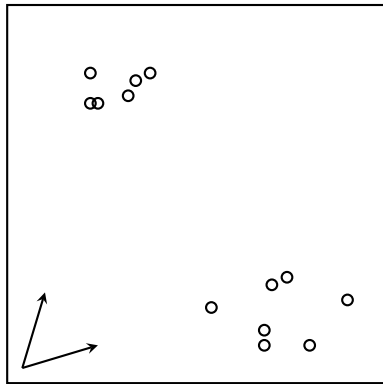
# Unsupervised learning

## Goal

Discover patterns/structure in  $X$ ,

## Unsupervised learning

- Unsupervised = no expert, no labels
- Main approaches:
  - Clustering = find a partition of  $X$  in  $K$  subsets,
  - Decomposition using  $K$  vectors.
  - Manifold Learning.
- Applications :
  - Dimensionality reduction, Quantization
  - Visualization...



2026-08-31

## Course 3: Unsupervised Learning

### └ Unsupervised learning

Unsupervised learning

Goal:  
Discover patterns/structure in  $X$ .

Unsupervised learning

- Unsupervised = no expert, no labels
- Main approaches:
  - Clustering = find a partition of  $X$  in  $K$  subsets,
  - Decomposition using  $K$  vectors.
  - Manifold Learning.
- Applications :
  - Dimensionality reduction, Quantization
  - Visualization

Three ways of saying "there is structure in this cloud".

Clustering answers *which points go together*: the answer is one integer per point. Decomposition answers *along which directions the cloud lives*: a few coordinates per point, in a basis estimated from the data. Manifold learning answers *what is the intrinsic shape of the cloud*: here the points are 2D but all lie close to a curve, so a single curvilinear coordinate almost describes them.

All three reduce the amount of information needed to describe a point. This is why the same two applications keep coming back: compression and visualization.

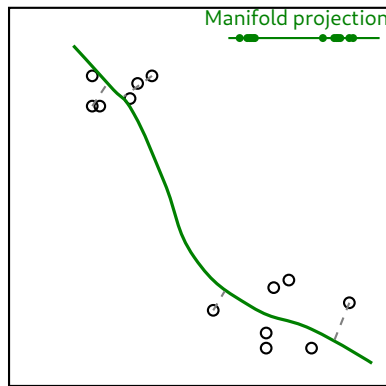
# Unsupervised learning

## Goal

Discover patterns/structure in  $X$ ,

## Unsupervised learning

- Unsupervised = no expert, no labels
- Main approaches:
  - Clustering = find a partition of  $X$  in  $K$  subsets,
  - Decomposition using  $K$  vectors.
  - Manifold Learning.
- Applications :
  - Dimensionality reduction, Quantization
  - Visualization...



2026-08-31

## Course 3: Unsupervised Learning

### └ Unsupervised learning

Unsupervised learning

Goal:  
Discover patterns/structure in  $X$ ,

Unsupervised learning

- Unsupervised = no expert, no labels
- Main approaches:
  - Clustering = find a partition of  $X$  in  $K$  subsets,
  - Decomposition using  $K$  vectors.
  - Manifold Learning.
- Applications :
  - Dimensionality reduction, Quantization
  - Visualization



Three ways of saying "there is structure in this cloud".

Clustering answers *which points go together*: the answer is one integer per point. Decomposition answers *along which directions the cloud lives*: a few coordinates per point, in a basis estimated from the data. Manifold learning answers *what is the intrinsic shape of the cloud*: here the points are 2D but all lie close to a curve, so a single curvilinear coordinate almost describes them.

All three reduce the amount of information needed to describe a point. This is why the same two applications keep coming back: compression and visualization.

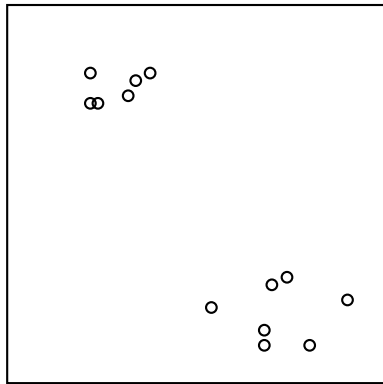
# Unsupervised learning

## Goal

Discover patterns/structure in  $X$ ,

## Unsupervised learning

- Unsupervised = no expert, no labels
- Main approaches:
  - Clustering = find a partition of  $X$  in  $K$  subsets,
  - Decomposition using  $K$  vectors.
  - Manifold Learning.
- Applications :
  - Dimensionality reduction, Quantization
  - Visualization...



2026-08-31

## Course 3: Unsupervised Learning

### └ Unsupervised learning

Unsupervised learning

Goal:  
Discover patterns/structure in  $X$ .

Unsupervised learning

- Unsupervised = no expert, no labels
- Main approaches:
  - Clustering = find a partition of  $X$  in  $K$  subsets,
  - Decomposition using  $K$  vectors.
  - Manifold Learning.
- Applications :
  - Dimensionality reduction, Quantization
  - Visualization...

Three ways of saying "there is structure in this cloud".

Clustering answers *which points go together*: the answer is one integer per point. Decomposition answers *along which directions the cloud lives*: a few coordinates per point, in a basis estimated from the data. Manifold learning answers *what is the intrinsic shape of the cloud*: here the points are 2D but all lie close to a curve, so a single curvilinear coordinate almost describes them.

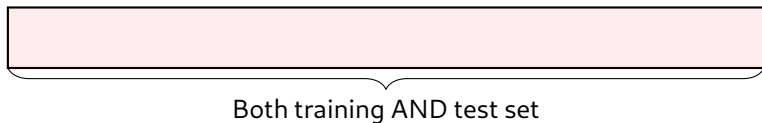
All three reduce the amount of information needed to describe a point. This is why the same two applications keep coming back: compression and visualization.

# Training and Test sets?

Full Dataset in train/test sets (as for supervised):



...Or actually:



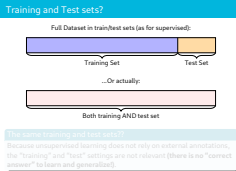
## The same training and test sets??

Because unsupervised learning does not rely on external annotations, the "training" and "test" settings are not relevant (**there is no "correct answer" to learn and generalize!**).

2026-08-31

## Course 3: Unsupervised Learning

### └ Training and Test sets?



One of the main practical differences with last session, and a classical source of confusion: since no label is used, nothing prevents you from fitting the model on all the data you have.

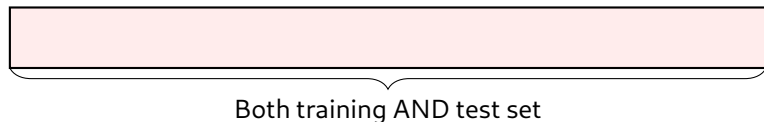
Two nuances. `fit` and `transform` remain separate in scikit-learn, so a fitted model can be applied to new points (projecting a new image with an existing PCA, for instance). This matters when unsupervised learning is only a pre-processing step before a supervised model: fit it on the training set only, otherwise information leaks from the test set. And some methods, TSNE in particular, have no `transform` at all; they only embed the points they were given.

# Training and Test sets?

Full Dataset in train/test sets (as for supervised):



...Or actually:



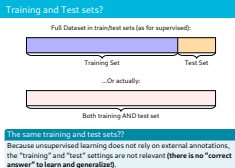
## The same training and test sets??

Because unsupervised learning does not rely on external annotations, the "training" and "test" settings are not relevant (**there is no "correct answer" to learn and generalize!**).

2026-08-31

## Course 3: Unsupervised Learning

### └ Training and Test sets?



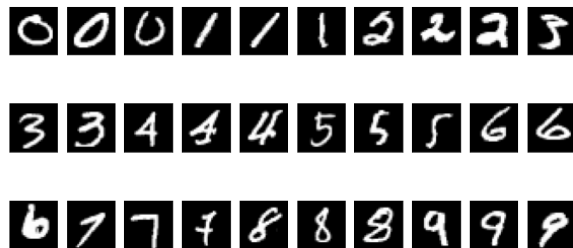
One of the main practical differences with last session, and a classical source of confusion: since no label is used, nothing prevents you from fitting the model on all the data you have.

Two nuances. `fit` and `transform` remain separate in scikit-learn, so a fitted model *can* be applied to new points (projecting a new image with an existing PCA, for instance). This matters when unsupervised learning is only a pre-processing step before a supervised model: fit it on the training set only, otherwise information leaks from the test set. And some methods, TSNE in particular, have no `transform` at all; they only embed the points they were given.

# A classical dataset: MNIST dataset (1/2)

## MNIST Dataset

- "Toy" dataset (=small and easy)
- 60000 + 10000 handwritten digits

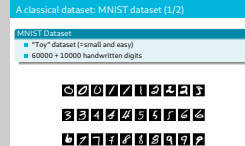


2026-08-31

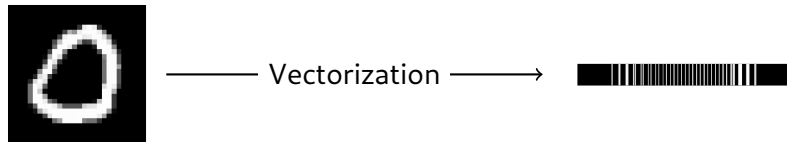
## Course 3: Unsupervised Learning

### └ A classical dataset: MNIST dataset (1/2)

A dataset that looks a little more like real data. MNIST is a small dataset of handwritten digits: 60000 examples for training and 10000 for test. It used to be an important benchmark, but it is considered far too easy today, hence the name "toy" dataset.



# A classical dataset: MNIST dataset (2/2)



Hence, all images are interpreted as 1D vectors!

2026-08-31

## Course 3: Unsupervised Learning

### A classical dataset: MNIST dataset (2/2)



Hence, all images are interpreted as 1D vectors!

This conditions everything that follows: a  $28 \times 28$  image becomes a vector of  $d = 784$  numbers, and the algorithm knows nothing about the 2D layout of the pixels. Two images are "close" if they have similar grey levels at the *same* pixel positions.

Ask yourself what happens if all the digits are shifted two pixels to the right. They look identical to you, but their vectors are now far apart. This is a strong limitation of the methods seen today, and one of the motivations for the representations of the next sessions.



# Plan

- 1 Clustering
- 2 Decomposition
- 3 Manifold learning
- 4 Evaluating unsupervised models
- 5 A note on data cleaning
- 6 Lab session

2026-08-31

## Course 3: Unsupervised Learning

└ Clustering

└ Plan

Plan

■ Clustering

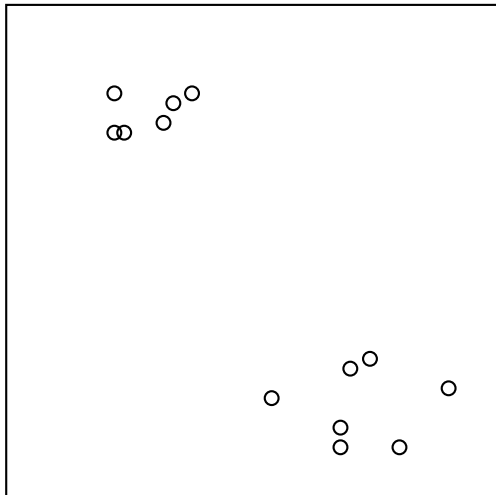
■ Decomposition

■ Manifold learning

■ Evaluating unsupervised models

■ A note on data cleaning

■ Lab session



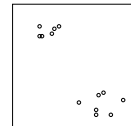
2026-08-31

Course 3: Unsupervised Learning

└ Clustering

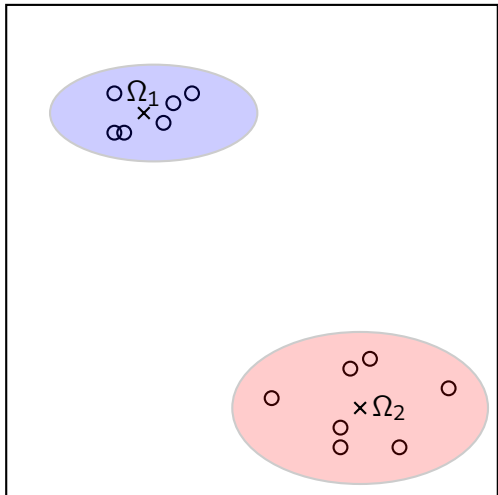
└ Clustering: the intuition

Clustering: the intuition



Here is a visual example. If we have the following set of points, then the two centroids  $\Omega_1$  and  $\Omega_2$  would be reasonable candidates for a clustering with two clusters.

# Clustering: the intuition



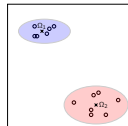
2026-08-31

Course 3: Unsupervised Learning

└ Clustering

└ Clustering: the intuition

Clustering: the intuition



Here is a visual example. If we have the following set of points, then the two centroids  $\Omega_1$  and  $\Omega_2$  would be reasonable candidates for a clustering with two clusters.

# A formal definition of clustering

An example to perform clustering is to rely on distances to centroids. We define  $K$  cluster centroids  $\Omega_k, \forall k \in [1..K]$ . Here, each vector is associated with the cluster whose centroid is of minimal distance.

## Definitions

We denote  $q : \mathbb{R}^d \rightarrow [1..K]$  a function that associates a vector  $\mathbf{x}$  with the index of (one of) its closest centroid  $\Omega_{q(\mathbf{x})}$ . Formally:

- $\forall k \in [1..K], \Omega_k \in \mathbb{R}^d$
- $\forall \mathbf{x} \in X, \forall j \in [1..K], \|\mathbf{x} - \Omega_{q(\mathbf{x})}\|_2 \leq \|\mathbf{x} - \Omega_j\|_2$
- Error (inertia)  $E(q) \triangleq \sum_{\mathbf{x} \in X} \|\mathbf{x} - \Omega_{q(\mathbf{x})}\|_2^2$
- $X = \bigcup_k \underbrace{\{\mathbf{x} \in X, q(\mathbf{x}) = k\}}_{\text{cluster } k}$

2026-08-31

## Course 3: Unsupervised Learning

### └ Clustering

#### └ A formal definition of clustering

#### A formal definition of clustering

An example to perform clustering is to rely on distances to centroids. We define  $K$  cluster centroids  $\Omega_k, \forall k \in [1..K]$ . Here, each vector is associated with the cluster whose centroid is of minimal distance.

#### Definitions

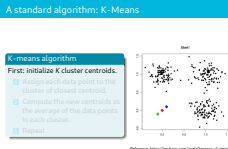
We denote  $q : \mathbb{R}^d \rightarrow [1..K]$  a function that associates a vector  $\mathbf{x}$  with the index of (one of) its closest centroid  $\Omega_{q(\mathbf{x})}$ . Formally:

- $\forall k \in [1..K], \Omega_k \in \mathbb{R}^d$
- $\forall \mathbf{x} \in X, \forall j \in [1..K], \|\mathbf{x} - \Omega_{q(\mathbf{x})}\|_2 \leq \|\mathbf{x} - \Omega_j\|_2$
- Error (inertia)  $E(q) \triangleq \sum_{\mathbf{x} \in X} \|\mathbf{x} - \Omega_{q(\mathbf{x})}\|_2^2$
- $X = \bigcup_k \underbrace{\{\mathbf{x} \in X, q(\mathbf{x}) = k\}}_{\text{cluster } k}$

A formal definition of clustering using centroids. Other definitions exist, based on regions, on density, or on probabilities.

The point to remember is the error: the sum of the *squared* distances between points and their closest centroid. scikit-learn calls it *inertia* (`model.inertia_`), and you will reuse it to choose  $K$ .

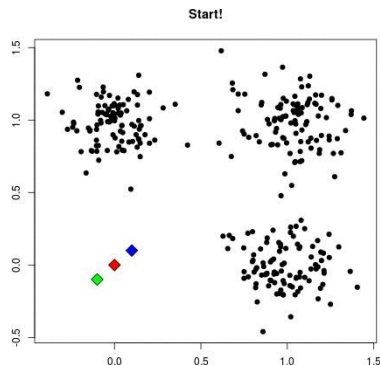
Note that it is expressed in the units of your features, squared: comparing it between two datasets is meaningless! Only compare it between two values of  $K$  on the same dataset.



## K-means algorithm

First: initialize  $K$  cluster centroids.

- 1 Assign each data point to the cluster of closest centroid.
- 2 Compute the new centroids as the average of the data points in each cluster.
- 3 Repeat.



Reference: <https://mubaris.com/posts/kmeans-clustering/>

Follow the animation: assign, then recompute, and repeat. Each step can only decrease the error, and there is a finite number of possible assignments, so the algorithm always converges, but to a *local* minimum.

Two consequences for the lab. The result depends on the initialization, which is why scikit-learn runs the algorithm several times (`n_init`) and keeps the best one. And since the algorithm only manipulates means and distances, it implicitly looks for round, similarly sized clusters: on elongated or nested shapes it will fail whatever you tune. That is where density-based methods such as DBSCAN become relevant.

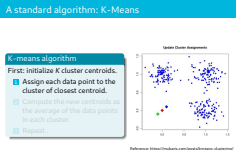
# A standard algorithm: K-Means

2026-08-31

## Course 3: Unsupervised Learning

### └ Clustering

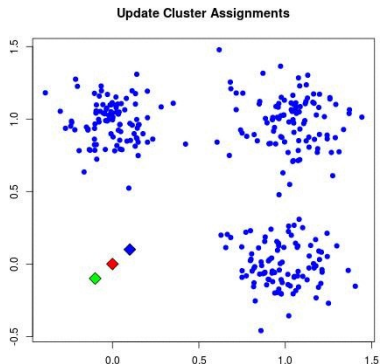
### └ A standard algorithm: K-Means



### K-means algorithm

First: initialize  $K$  cluster centroids.

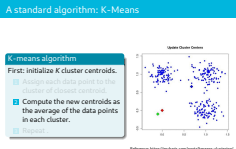
- 1 Assign each data point to the cluster of closest centroid.
- 2 Compute the new centroids as the average of the data points in each cluster.
- 3 Repeat.



Reference: <https://mubaris.com/posts/kmeans-clustering/>

Follow the animation: assign, then recompute, and repeat. Each step can only decrease the error, and there is a finite number of possible assignments, so the algorithm always converges, but to a *local* minimum.

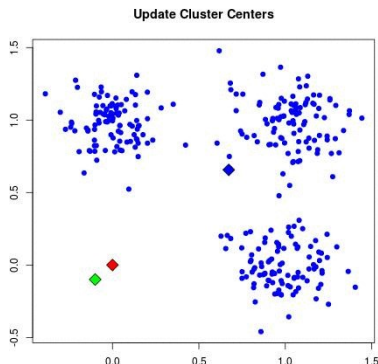
Two consequences for the lab. The result depends on the initialization, which is why scikit-learn runs the algorithm several times (`n_init`) and keeps the best one. And since the algorithm only manipulates means and distances, it implicitly looks for round, similarly sized clusters: on elongated or nested shapes it will fail whatever you tune. That is where density-based methods such as DBSCAN become relevant.



## K-means algorithm

First: initialize  $K$  cluster centroids.

- 1 Assign each data point to the cluster of closest centroid.
- 2 Compute the new centroids as the average of the data points in each cluster.
- 3 Repeat .



Reference: <https://mubaris.com/posts/kmeans-clustering/>

Follow the animation: assign, then recompute, and repeat. Each step can only decrease the error, and there is a finite number of possible assignments, so the algorithm always converges, but to a *local* minimum.

Two consequences for the lab. The result depends on the initialization, which is why scikit-learn runs the algorithm several times (`n_init`) and keeps the best one. And since the algorithm only manipulates means and distances, it implicitly looks for round, similarly sized clusters: on elongated or nested shapes it will fail whatever you tune. That is where density-based methods such as DBSCAN become relevant.

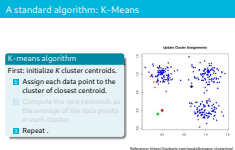
# A standard algorithm: K-Means

2026-08-31

## Course 3: Unsupervised Learning

### Clustering

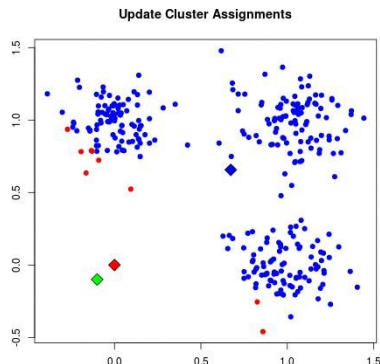
#### A standard algorithm: K-Means



### K-means algorithm

First: initialize  $K$  cluster centroids.

- 1 Assign each data point to the cluster of closest centroid.
- 2 Compute the new centroids as the average of the data points in each cluster.
- 3 Repeat .



Reference: <https://mubaris.com/posts/kmeans-clustering/>

Follow the animation: assign, then recompute, and repeat. Each step can only decrease the error, and there is a finite number of possible assignments, so the algorithm always converges, but to a *local* minimum.

Two consequences for the lab. The result depends on the initialization, which is why scikit-learn runs the algorithm several times (`n_init`) and keeps the best one. And since the algorithm only manipulates means and distances, it implicitly looks for round, similarly sized clusters: on elongated or nested shapes it will fail whatever you tune. That is where density-based methods such as DBSCAN become relevant.



# A standard algorithm: K-Means

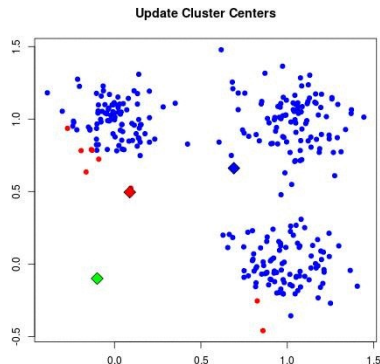
2026-08-31 Course 3: Unsupervised Learning  
└ Clustering

└ A standard algorithm: K-Means

## K-means algorithm

First: initialize  $K$  cluster centroids.

- 1 Assign each data point to the cluster of closest centroid.
- 2 Compute the new centroids as the average of the data points in each cluster.
- 3 Repeat .



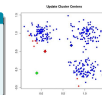
Reference: <https://mubaris.com/posts/kmeans-clustering/>

A standard algorithm: K-Means

### K-means algorithm

First: initialize  $K$  cluster centroids.

- 1 Assign each data point to the cluster of closest centroid.
- 2 Compute the new centroids as the average of the data points in each cluster.
- 3 Repeat .



Reference: <https://mubaris.com/posts/kmeans-clustering/>

Follow the animation: assign, then recompute, and repeat. Each step can only decrease the error, and there is a finite number of possible assignments, so the algorithm always converges, but to a *local* minimum.

Two consequences for the lab. The result depends on the initialization, which is why scikit-learn runs the algorithm several times (`n_init`) and keeps the best one. And since the algorithm only manipulates means and distances, it implicitly looks for round, similarly sized clusters: on elongated or nested shapes it will fail whatever you tune. That is where density-based methods such as DBSCAN become relevant.

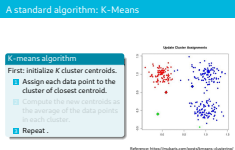
# A standard algorithm: K-Means

2026-08-31

## Course 3: Unsupervised Learning

### Clustering

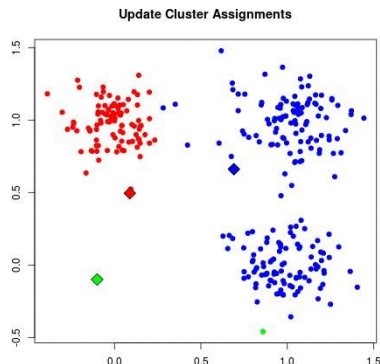
#### A standard algorithm: K-Means



### K-means algorithm

First: initialize  $K$  cluster centroids.

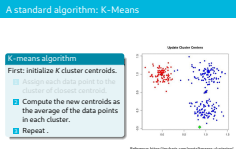
- 1 Assign each data point to the cluster of closest centroid.
- 2 Compute the new centroids as the average of the data points in each cluster.
- 3 Repeat .



Reference: <https://mubaris.com/posts/kmeans-clustering/>

Follow the animation: assign, then recompute, and repeat. Each step can only decrease the error, and there is a finite number of possible assignments, so the algorithm always converges, but to a *local* minimum.

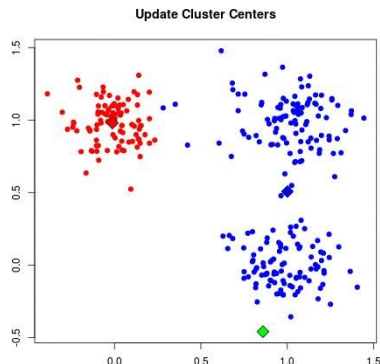
Two consequences for the lab. The result depends on the initialization, which is why scikit-learn runs the algorithm several times (`n_init`) and keeps the best one. And since the algorithm only manipulates means and distances, it implicitly looks for round, similarly sized clusters: on elongated or nested shapes it will fail whatever you tune. That is where density-based methods such as DBSCAN become relevant.



## K-means algorithm

First: initialize  $K$  cluster centroids.

- 1 Assign each data point to the cluster of closest centroid.
- 2 Compute the new centroids as the average of the data points in each cluster.
- 3 Repeat .



Reference: <https://mubaris.com/posts/kmeans-clustering/>

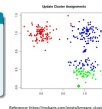
Follow the animation: assign, then recompute, and repeat. Each step can only decrease the error, and there is a finite number of possible assignments, so the algorithm always converges, but to a *local* minimum.

Two consequences for the lab. The result depends on the initialization, which is why scikit-learn runs the algorithm several times (`n_init`) and keeps the best one. And since the algorithm only manipulates means and distances, it implicitly looks for round, similarly sized clusters: on elongated or nested shapes it will fail whatever you tune. That is where density-based methods such as DBSCAN become relevant.

## K-means algorithm

First: initialize  $K$  cluster centroids.

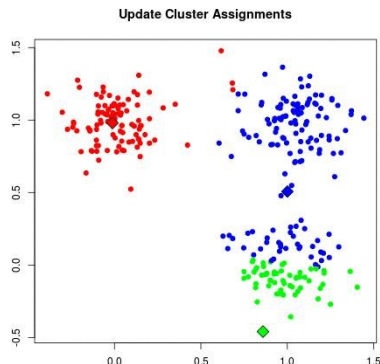
- 1 Assign each data point to the cluster of closest centroid.
- 2 Compute the new centroids as the average of the data points in each cluster.
- 3 Repeat .



## K-means algorithm

First: initialize  $K$  cluster centroids.

- 1 Assign each data point to the cluster of closest centroid.
- 2 Compute the new centroids as the average of the data points in each cluster.
- 3 Repeat .



Reference: <https://mubaris.com/posts/kmeans-clustering/>

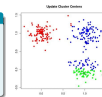
Follow the animation: assign, then recompute, and repeat. Each step can only decrease the error, and there is a finite number of possible assignments, so the algorithm always converges, but to a *local* minimum.

Two consequences for the lab. The result depends on the initialization, which is why scikit-learn runs the algorithm several times (`n_init`) and keeps the best one. And since the algorithm only manipulates means and distances, it implicitly looks for round, similarly sized clusters: on elongated or nested shapes it will fail whatever you tune. That is where density-based methods such as DBSCAN become relevant.

## K-means algorithm

First: initialize  $K$  cluster centroids.

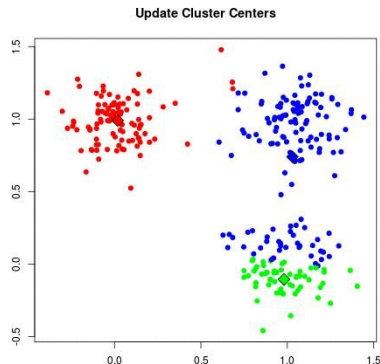
- 1 Assign each data point to the cluster of closest centroid.
- 2 Compute the new centroids as the average of the data points in each cluster.
- 3 Repeat .



## K-means algorithm

First: initialize  $K$  cluster centroids.

- 1 Assign each data point to the cluster of closest centroid.
- 2 Compute the new centroids as the average of the data points in each cluster.
- 3 Repeat .



Reference: <https://mubaris.com/posts/kmeans-clustering/>

Follow the animation: assign, then recompute, and repeat. Each step can only decrease the error, and there is a finite number of possible assignments, so the algorithm always converges, but to a *local* minimum.

Two consequences for the lab. The result depends on the initialization, which is why scikit-learn runs the algorithm several times (`n_init`) and keeps the best one. And since the algorithm only manipulates means and distances, it implicitly looks for round, similarly sized clusters: on elongated or nested shapes it will fail whatever you tune. That is where density-based methods such as DBSCAN become relevant.

# A standard algorithm: K-Means

2026-08-31

Course 3: Unsupervised Learning

└ Clustering

└ A standard algorithm: K-Means

A standard algorithm: K-Means

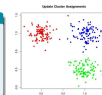
K-means algorithm

First: initialize  $K$  cluster centroids.

1 Assign each data point to the cluster of closest centroid.

2 Compute the new centroids as the average of the data points in each cluster.

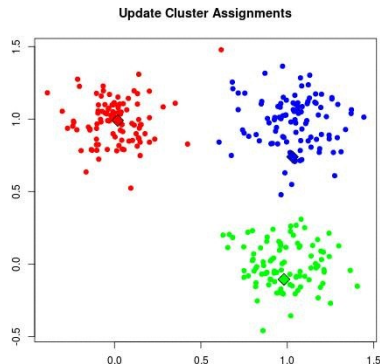
3 Repeat .



## K-means algorithm

First: initialize  $K$  cluster centroids.

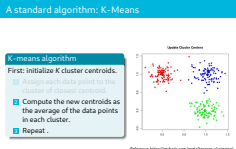
- 1 Assign each data point to the cluster of closest centroid.
- 2 Compute the new centroids as the average of the data points in each cluster.
- 3 Repeat .



Reference: <https://mubaris.com/posts/kmeans-clustering/>

Follow the animation: assign, then recompute, and repeat. Each step can only decrease the error, and there is a finite number of possible assignments, so the algorithm always converges, but to a *local* minimum.

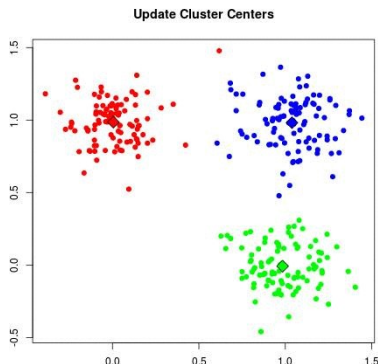
Two consequences for the lab. The result depends on the initialization, which is why scikit-learn runs the algorithm several times (`n_init`) and keeps the best one. And since the algorithm only manipulates means and distances, it implicitly looks for round, similarly sized clusters: on elongated or nested shapes it will fail whatever you tune. That is where density-based methods such as DBSCAN become relevant.



## K-means algorithm

First: initialize  $K$  cluster centroids.

- 1 Assign each data point to the cluster of closest centroid.
- 2 Compute the new centroids as the average of the data points in each cluster.
- 3 Repeat.



Reference: <https://mubaris.com/posts/kmeans-clustering/>

Follow the animation: assign, then recompute, and repeat. Each step can only decrease the error, and there is a finite number of possible assignments, so the algorithm always converges, but to a *local* minimum.

Two consequences for the lab. The result depends on the initialization, which is why scikit-learn runs the algorithm several times (`n_init`) and keeps the best one. And since the algorithm only manipulates means and distances, it implicitly looks for round, similarly sized clusters: on elongated or nested shapes it will fail whatever you tune. That is where density-based methods such as DBSCAN become relevant.

# A standard algorithm: K-Means

2026-08-31

## Course 3: Unsupervised Learning

### Clustering

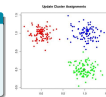
#### A standard algorithm: K-Means

A standard algorithm: K-Means

##### K-means algorithm

First: initialize  $K$  cluster centroids.

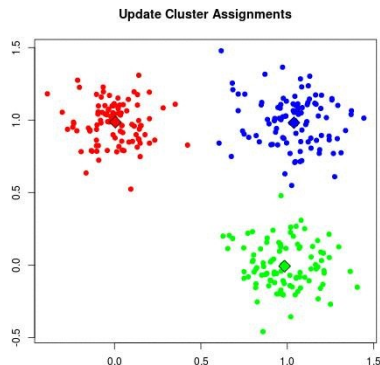
- 1 Assign each data point to the cluster of closest centroid.
- 2 Compute the new centroids as the average of the data points in each cluster.
- 3 Repeat until convergence.



### K-means algorithm

First: initialize  $K$  cluster centroids.

- 1 Assign each data point to the cluster of closest centroid.
- 2 Compute the new centroids as the average of the data points in each cluster.
- 3 Repeat until convergence.



Reference: <https://mubaris.com/posts/kmeans-clustering/>

Follow the animation: assign, then recompute, and repeat. Each step can only decrease the error, and there is a finite number of possible assignments, so the algorithm always converges, but to a *local* minimum.

Two consequences for the lab. The result depends on the initialization, which is why scikit-learn runs the algorithm several times (`n_init`) and keeps the best one. And since the algorithm only manipulates means and distances, it implicitly looks for round, similarly sized clusters: on elongated or nested shapes it will fail whatever you tune. That is where density-based methods such as DBSCAN become relevant.



# In practice: K-Means on MNIST

## Clustering MNIST

Using K-means algorithm with  $K = 10$ .

→ Clusters capture, to some extent, the different digits.

Original MNIST examples



Cluster centroids



Note: we recall that images are vectorized for the clustering to make sense! They are only displayed in 2D to be interpretable.

2026-08-31

## Course 3: Unsupervised Learning

### └ Clustering

#### └ In practice: K-Means on MNIST

In practice: K-Means on MNIST

#### Clustering MNIST

Using K-means algorithm with  $K = 10$ .

→ Clusters capture, to some extent, the different digits.

Original MNIST examples



Cluster centroids

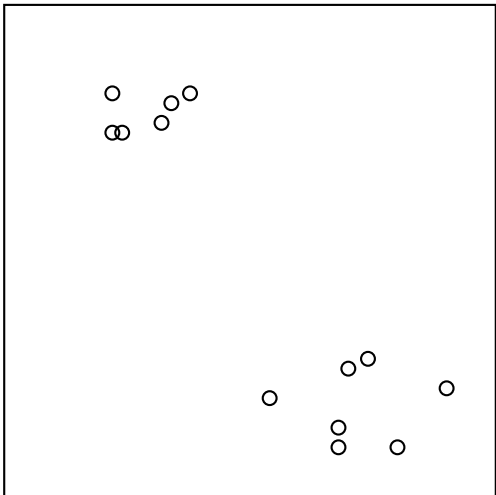


Note: we recall that images are vectorized for the clustering to make sense! They are only displayed in 2D to be interpretable.

A clustering test on MNIST with the K-means algorithm of the previous slide, run with  $K = 10$ .

On the left, original examples; on the right, the 10 centroids obtained. Some clusters clearly capture one digit (6, 1, 2, 0), but others split a single digit into several clusters (8, 4, 3). With  $K = 10$  there is no guarantee that the 10 clusters match the 10 classes; nothing in the algorithm knows that digits exist.

# Impact of the choice of $K$



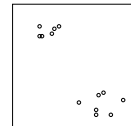
2026-08-31

## Course 3: Unsupervised Learning

### └ Clustering

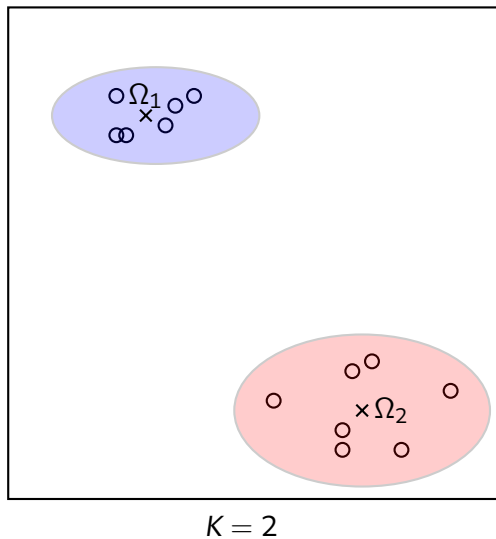
#### └ Impact of the choice of $K$

Impact of the choice of  $K$



Changing the number of centroids changes the clustering, and with it the meaning of the clusters. Look at the two extremes: with  $K = 1$  everything collapses into a single group, with  $K = N$  each point is its own cluster and nothing has been learned. Neither is useful, so  $K$  has to be chosen.

# Impact of the choice of $K$



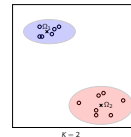
2026-08-31

## Course 3: Unsupervised Learning

### └ Clustering

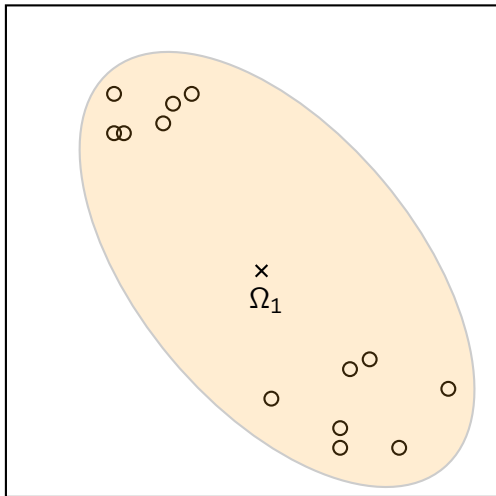
#### └ Impact of the choice of $K$

Impact of the choice of  $K$



Changing the number of centroids changes the clustering, and with it the meaning of the clusters. Look at the two extremes: with  $K = 1$  everything collapses into a single group, with  $K = N$  each point is its own cluster and nothing has been learned. Neither is useful, so  $K$  has to be chosen.

# Impact of the choice of $K$



$K = 1$

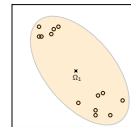
2026-08-31

Course 3: Unsupervised Learning

└ Clustering

└ Impact of the choice of  $K$

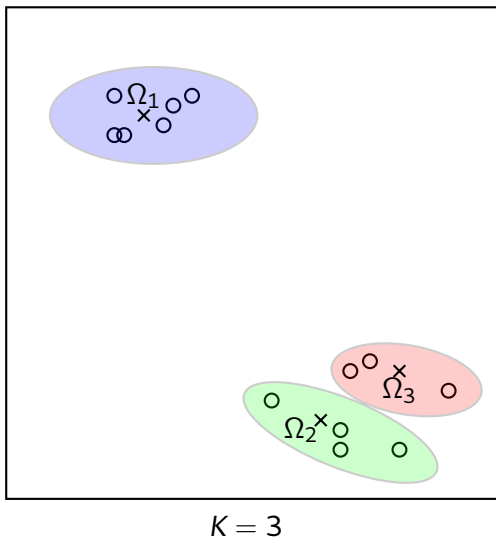
Impact of the choice of  $K$



$K = 1$

Changing the number of centroids changes the clustering, and with it the meaning of the clusters. Look at the two extremes: with  $K = 1$  everything collapses into a single group, with  $K = N$  each point is its own cluster and nothing has been learned. Neither is useful, so  $K$  has to be chosen.

# Impact of the choice of $K$



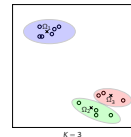
2026-08-31

## Course 3: Unsupervised Learning

### └ Clustering

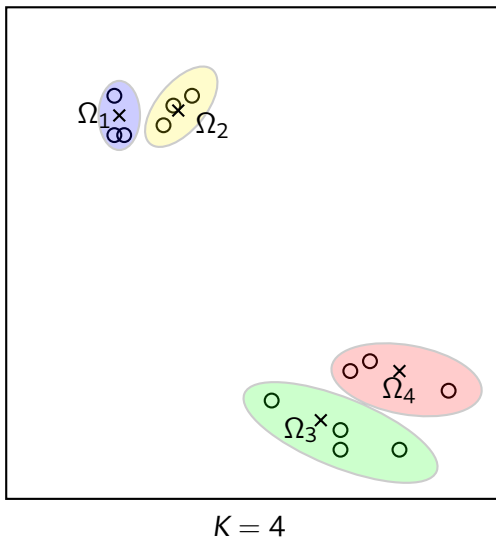
#### └ Impact of the choice of $K$

Impact of the choice of  $K$



Changing the number of centroids changes the clustering, and with it the meaning of the clusters. Look at the two extremes: with  $K = 1$  everything collapses into a single group, with  $K = N$  each point is its own cluster and nothing has been learned. Neither is useful, so  $K$  has to be chosen.

# Impact of the choice of $K$



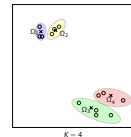
2026-08-31

## Course 3: Unsupervised Learning

### └ Clustering

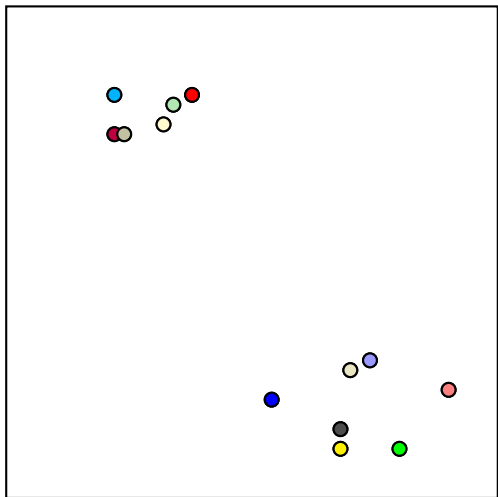
#### └ Impact of the choice of $K$

Impact of the choice of  $K$



Changing the number of centroids changes the clustering, and with it the meaning of the clusters. Look at the two extremes: with  $K = 1$  everything collapses into a single group, with  $K = N$  each point is its own cluster and nothing has been learned. Neither is useful, so  $K$  has to be chosen.

# Impact of the choice of $K$



$K = N$   
(each data point is its own centroid)

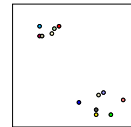
2026-08-31

Course 3: Unsupervised Learning

└ Clustering

└ Impact of the choice of  $K$

Impact of the choice of  $K$



$K = N$   
(each data point is its own centroid)

Changing the number of centroids changes the clustering, and with it the meaning of the clusters. Look at the two extremes: with  $K = 1$  everything collapses into a single group, with  $K = N$  each point is its own cluster and nothing has been learned. Neither is useful, so  $K$  has to be chosen.

# Choosing $K$

## Impact of $K$

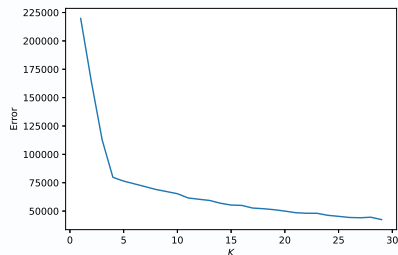
Changing the number of centroids changes the clustering...

→ **And the significance of clusters!**

Choosing  $K$ : compromise between **error** and **compression** ( $\kappa = 1 - \frac{K}{N}$ : the fewer clusters, the more we compress).

## "Elbow" heuristic

Not always clear... May be a good starting point!



2026-08-31

## Course 3: Unsupervised Learning

### └ Clustering

### └ Choosing $K$

The compression factor  $\kappa$  makes the compromise explicit: a large  $K$  gives a small error but almost no compression ( $\kappa \rightarrow 0$ ), a small  $K$  the opposite. Choosing  $K$  is choosing a point on that trade-off.

The elbow plot is the ideal case: there is a clear value of  $K$  after which adding clusters barely reduces the error. On real data the bend is often much less obvious, so do not be surprised if you cannot find one.

#### Choosing $K$

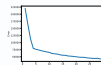
##### Impact of $K$

Changing the number of centroids changes the clustering...  
→ **And the significance of clusters!**

Choosing  $K$ : compromise between **error** and **compression** ( $\kappa = 1 - \frac{K}{N}$ : the fewer clusters, the more we compress).

##### "Elbow" heuristic

Not always clear... May be a good starting point!





# Choosing $K$

## Impact of $K$

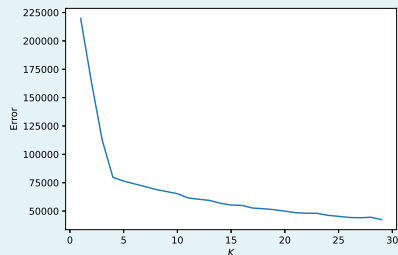
Changing the number of centroids changes the clustering...

→ **And the significance of clusters!**

Choosing  $K$ : compromise between **error** and **compression** ( $\kappa = 1 - \frac{K}{N}$ : the fewer clusters, the more we compress).

## "Elbow" heuristic

Not always clear... May be a good starting point!



2026-08-31

## Course 3: Unsupervised Learning

### └ Clustering

### └ Choosing $K$

The compression factor  $\kappa$  makes the compromise explicit: a large  $K$  gives a small error but almost no compression ( $\kappa \rightarrow 0$ ), a small  $K$  the opposite. Choosing  $K$  is choosing a point on that trade-off.

The elbow plot is the ideal case: there is a clear value of  $K$  after which adding clusters barely reduces the error. On real data the bend is often much less obvious, so do not be surprised if you cannot find one.

Choosing  $K$

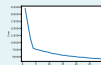
Impact of  $K$

Changing the number of centroids changes the clustering...  
→ **And the significance of clusters!**

Choosing  $K$ : compromise between **error** and **compression** ( $\kappa = 1 - \frac{K}{N}$ : the fewer clusters, the more we compress).

"Elbow" heuristic:

Not always clear... May be a good starting point!



# Optimal clustering is not tractable!

## Optimal clustering

- Optimal clustering:  $q^* \triangleq \arg \min_{q: \mathbb{R}^d \rightarrow [1..K]} E(q)$ , and  $E_{opt_K} \triangleq E(q^*)$ ,
- By construction, increasing  $K$  can only decrease the optimal error:  
 $0 = E_{opt_N} \leq E_{opt_{N-1}} \leq \dots \leq E_{opt_1} = N \cdot \text{var}(X)$
- Finding an optimal clustering is an NP-hard problem!

2026-08-31

## Course 3: Unsupervised Learning

### └ Clustering

### └ Optimal clustering is not tractable!

#### Optimal clustering

- Optimal clustering:  $q^* \triangleq \arg \min_{q: \mathbb{R}^d \rightarrow [1..K]} E(q)$ , and  $E_{opt_K} \triangleq E(q^*)$ .
- By construction, increasing  $K$  can only decrease the optimal error:  
 $0 = E_{opt_N} \leq E_{opt_{N-1}} \leq \dots \leq E_{opt_1} = N \cdot \text{var}(X)$
- Finding an optimal clustering is an NP-hard problem!

About the properties.

If you take one centroid per point ( $K = N$ ), the error is obviously 0. If you take a single cluster, the best centroid is the average of all points, and the error is exactly  $N$  times the variance of  $X$  (with  $E$  defined as a *sum* of squared distances). The optimal error therefore never increases with  $K$ , so it can never be used alone to choose  $K$ : the “best” value would always be  $K = N$ . This is exactly the trade-off measured by the compression factor  $\kappa$  of the previous slide, and the reason the elbow heuristic exists.

One word on NP-hardness in K-means: we never look for the optimum, only for a decent local solution. Hence, the result depends on the initialization.

# Plan

1 Clustering

2 Decomposition

3 Manifold learning

4 Evaluating unsupervised models

5 A note on data cleaning

6 Lab session

2026-08-31

## Course 3: Unsupervised Learning

└─ Decomposition

└─ Plan

Plan

■ Clustering

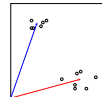
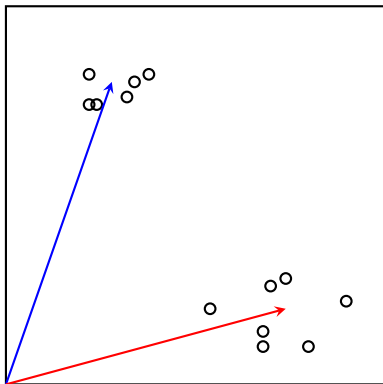
■ Decomposition

■ Manifold learning

■ Evaluating unsupervised models

■ A note on data cleaning

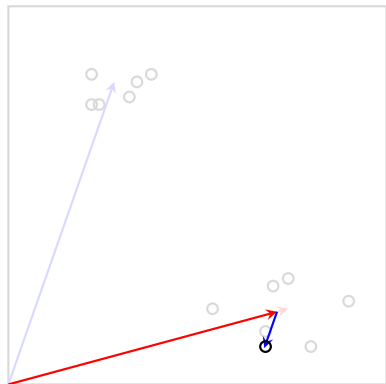
■ Lab session



A change of point of view. Instead of replacing a point by the index of its cluster, we replace it by a short list of coefficients: how much of component 1, how much of component 2.

Each point is rewritten as a weighted sum of the two components (red and blue). Points of the bottom-right group have a large red weight, those of the top-left group a large blue weight: the weights already carry the structure of the data.

The last overlay gathers this in matrix form,  $X \approx UV$ . The rows of  $V$  are the  $K$  components, shared by all points; the rows of  $U$  are the new coordinates, one per point. Storing  $U$  and  $V$  instead of  $X$  is a compression as soon as  $K(N + d) < Nd$ .



$$x = 0.96 \times \text{red arrow} + -0.12 \times \text{blue arrow}$$

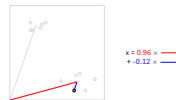
2026-08-31

## Course 3: Unsupervised Learning

### └ Decomposition

### └ Decomposition

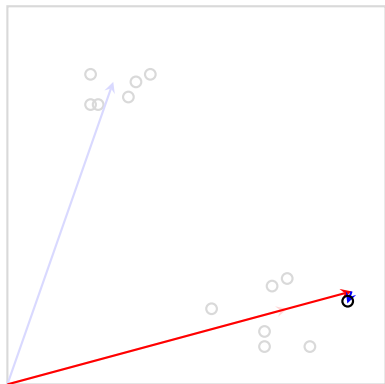
Decomposition



A change of point of view. Instead of replacing a point by the index of its cluster, we replace it by a short list of coefficients: how much of component 1, how much of component 2.

Each point is rewritten as a weighted sum of the two components (red and blue). Points of the bottom-right group have a large red weight, those of the top-left group a large blue weight: the weights already carry the structure of the data.

The last overlay gathers this in matrix form,  $X \approx UV$ . The rows of  $V$  are the  $K$  components, shared by all points; the rows of  $U$  are the new coordinates, one per point. Storing  $U$  and  $V$  instead of  $X$  is a compression as soon as  $K(N + d) < Nd$ .



$$x = 1.23 \times \text{red arrow} + -0.04 \times \text{blue arrow}$$

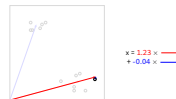
2026-08-31

## Course 3: Unsupervised Learning

### └ Decomposition

#### └ Decomposition

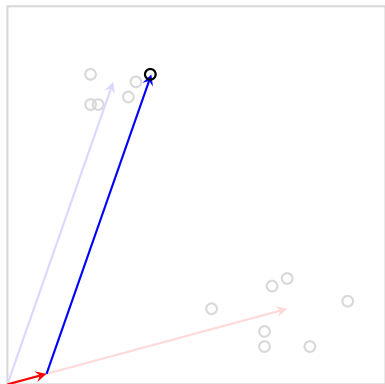
Decomposition



A change of point of view. Instead of replacing a point by the index of its cluster, we replace it by a short list of coefficients: how much of component 1, how much of component 2.

Each point is rewritten as a weighted sum of the two components (red and blue). Points of the bottom-right group have a large red weight, those of the top-left group a large blue weight: the weights already carry the structure of the data.

The last overlay gathers this in matrix form,  $X \approx UV$ . The rows of  $V$  are the  $K$  components, shared by all points; the rows of  $U$  are the new coordinates, one per point. Storing  $U$  and  $V$  instead of  $X$  is a compression as soon as  $K(N + d) < Nd$ .



$$x = 0.14 \times \text{red arrow} + 0.99 \times \text{blue arrow}$$

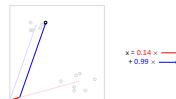
2026-08-31

## Course 3: Unsupervised Learning

### └ Decomposition

### └ Decomposition

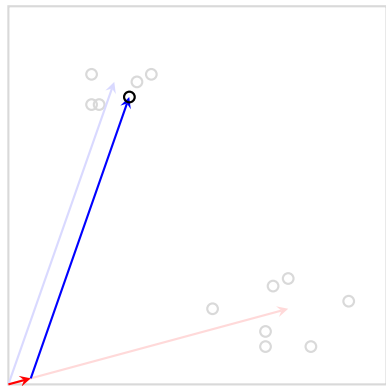
Decomposition



A change of point of view. Instead of replacing a point by the index of its cluster, we replace it by a short list of coefficients: how much of component 1, how much of component 2.

Each point is rewritten as a weighted sum of the two components (red and blue). Points of the bottom-right group have a large red weight, those of the top-left group a large blue weight: the weights already carry the structure of the data.

The last overlay gathers this in matrix form,  $X \approx UV$ . The rows of  $V$  are the  $K$  components, shared by all points; the rows of  $U$  are the new coordinates, one per point. Storing  $U$  and  $V$  instead of  $X$  is a compression as soon as  $K(N + d) < Nd$ .



$$\mathbf{x} = 0.08 \times \text{red arrow} + 0.93 \times \text{blue arrow}$$

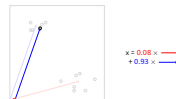
2026-08-31

## Course 3: Unsupervised Learning

### └ Decomposition

#### └ Decomposition

Decomposition

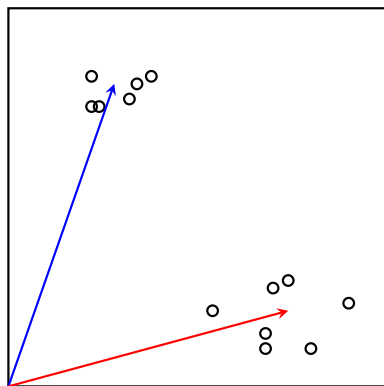


A change of point of view. Instead of replacing a point by the index of its cluster, we replace it by a short list of coefficients: how much of component 1, how much of component 2.

Each point is rewritten as a weighted sum of the two components (red and blue). Points of the bottom-right group have a large red weight, those of the top-left group a large blue weight: the weights already carry the structure of the data.

The last overlay gathers this in matrix form,  $X \approx UV$ . The rows of  $V$  are the  $K$  components, shared by all points; the rows of  $U$  are the new coordinates, one per point. Storing  $U$  and  $V$  instead of  $X$  is a compression as soon as  $K(N + d) < Nd$ .





$$\approx \begin{matrix} U & V \\ \text{"weights"} & \text{"Components"} \end{matrix}$$

|      |       |
|------|-------|
| 0.96 | -0.12 |
| 1.23 | -0.04 |
| ...  |       |
| 0.14 | 0.99  |
| 0.08 | 0.93  |
| ...  |       |

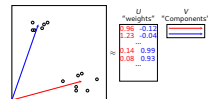
2026-08-31

## Course 3: Unsupervised Learning

### Decomposition

### Decomposition

Decomposition



A change of point of view. Instead of replacing a point by the index of its cluster, we replace it by a short list of coefficients: how much of component 1, how much of component 2.

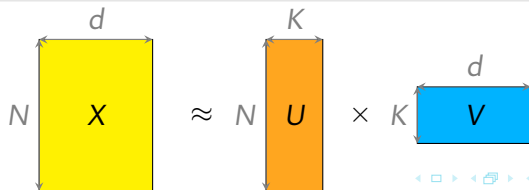
Each point is rewritten as a weighted sum of the two components (red and blue). Points of the bottom-right group have a large red weight, those of the top-left group a large blue weight: the weights already carry the structure of the data.

The last overlay gathers this in matrix form,  $X \approx UV$ . The rows of  $V$  are the  $K$  components, shared by all points; the rows of  $U$  are the new coordinates, one per point. Storing  $U$  and  $V$  instead of  $X$  is a compression as soon as  $K(N + d) < Nd$ .

## Definitions

Principal components analysis solves the following matrix factorization problem:

- The set  $X$  is considered as a matrix  $X \in \mathcal{M}_{N \times d}(\mathbb{R})$ ,
- We consider decompositions using components  $V \in \mathcal{M}_{K \times d}(\mathbb{R})$  and weights  $U \in \mathcal{M}_{N \times K}(\mathbb{R})$ ,
- PCA estimates  $K$  components that are orthogonal and ordered by importance (variance explained)
- It is based on the Singular Value Decomposition (SVD) of the covariance matrix  $XX^T$



2026-08-31

## Course 3: Unsupervised Learning

### └ Decomposition

### └ Principal Components Analysis

Principal Components Analysis

Definitions

Principal components analysis solves the following matrix factorization problem:

- The set  $X$  is considered as a matrix  $X \in \mathcal{M}_{N \times d}(\mathbb{R})$ ,
- We consider decompositions using components  $V \in \mathcal{M}_{K \times d}(\mathbb{R})$  and weights  $U \in \mathcal{M}_{N \times K}(\mathbb{R})$ ,
- PCA estimates  $K$  components that are orthogonal and ordered by importance (variance explained)
- It is based on the Singular Value Decomposition (SVD) of the covariance matrix  $XX^T$

The small diagram shows a yellow rectangle  $X$  with dimensions  $N$  and  $d$ , followed by an approximation symbol  $\approx$ , then an orange rectangle  $U$  with dimensions  $N$  and  $K$ , followed by a multiplication symbol  $\times$ , then a blue rectangle  $V$  with dimensions  $K$  and  $d$ .

PCA is the simplest and by far the most used member of this family. Among all the ways of writing  $X \approx UV$  with  $K$  components, it picks the one minimising the reconstruction error  $\|X - UV\|^2$ , with components orthogonal and ordered by decreasing explained variance.

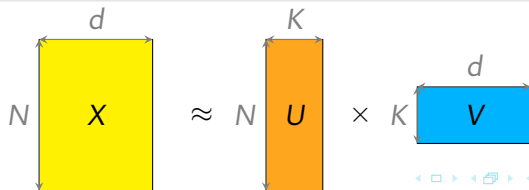
Two consequences. The components are nested: the first two of a  $K = 32$  decomposition are the same as those of a  $K = 2$  one, which is why PCA is one of the default tools for 2D visualization. And `explained_variance_ratio_` tells you how much of the data you keep; its cumulative sum thus provides a way to choose  $K$ .

Warning: PCA is variance-based, so run it on scaled/normalized features, otherwise the feature with the largest units wins.

## Definitions

Principal components analysis solves the following matrix factorization problem:

- The set  $X$  is considered as a matrix  $X \in \mathcal{M}_{N \times d}(\mathbb{R})$ ,
- We consider decompositions using components  $V \in \mathcal{M}_{K \times d}(\mathbb{R})$  and weights  $U \in \mathcal{M}_{N \times K}(\mathbb{R})$ ,
- PCA estimates  $K$  components that are orthogonal and ordered by importance (variance explained)
- It is based on the Singular Value Decomposition (SVD) of the covariance matrix  $XX^T$



2026-08-31

## Course 3: Unsupervised Learning

### └ Decomposition

### └ Principal Components Analysis

Principal Components Analysis

Definitions

Principal components analysis solves the following matrix factorization problem:

- The set  $X$  is considered as a matrix  $X \in \mathcal{M}_{N \times d}(\mathbb{R})$ ,
- We consider decompositions using components  $V \in \mathcal{M}_{K \times d}(\mathbb{R})$  and weights  $U \in \mathcal{M}_{N \times K}(\mathbb{R})$ ,
- PCA estimates  $K$  components that are orthogonal and ordered by importance (variance explained)
- It is based on the Singular Value Decomposition (SVD) of the covariance matrix  $XX^T$

The diagram shows the matrix factorization  $X \approx UV$ . Matrix  $X$  is a yellow rectangle with height  $N$  and width  $d$ . Matrix  $U$  is an orange rectangle with height  $N$  and width  $K$ . Matrix  $V$  is a blue rectangle with height  $K$  and width  $d$ . The equation  $X \approx UV$  is shown with the matrices and their dimensions.

PCA is the simplest and by far the most used member of this family. Among all the ways of writing  $X \approx UV$  with  $K$  components, it picks the one minimising the reconstruction error  $\|X - UV\|^2$ , with components orthogonal and ordered by decreasing explained variance.

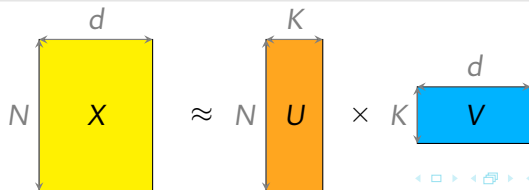
Two consequences. The components are nested: the first two of a  $K = 32$  decomposition are the same as those of a  $K = 2$  one, which is why PCA is one of the default tools for 2D visualization. And `explained_variance_ratio_` tells you how much of the data you keep; its cumulative sum thus provides a way to choose  $K$ .

Warning: PCA is variance-based, so run it on scaled/normalized features, otherwise the feature with the largest units wins.

## Definitions

Principal components analysis solves the following matrix factorization problem:

- The set  $X$  is considered as a matrix  $X \in \mathcal{M}_{N \times d}(\mathbb{R})$ ,
- We consider decompositions using components  $V \in \mathcal{M}_{K \times d}(\mathbb{R})$  and weights  $U \in \mathcal{M}_{N \times K}(\mathbb{R})$ ,
- PCA estimates  $K$  components that are orthogonal and ordered by importance (variance explained)
- It is based on the Singular Value Decomposition (SVD) of the covariance matrix  $XX^T$



2026-08-31

## Course 3: Unsupervised Learning

### └ Decomposition

### └ Principal Components Analysis

Principal Components Analysis

Definitions

Principal components analysis solves the following matrix factorization problem:

- The set  $X$  is considered as a matrix  $X \in \mathcal{M}_{N \times d}(\mathbb{R})$ ,
- We consider decompositions using components  $V \in \mathcal{M}_{K \times d}(\mathbb{R})$  and weights  $U \in \mathcal{M}_{N \times K}(\mathbb{R})$ ,
- PCA estimates  $K$  components that are orthogonal and ordered by importance (variance explained)
- It is based on the Singular Value Decomposition (SVD) of the covariance matrix  $XX^T$

The small diagram shows a yellow rectangle  $X$  with dimensions  $N$  and  $d$ , followed by an approximation symbol  $\approx$ , then an orange rectangle  $U$  with dimensions  $N$  and  $K$ , followed by a multiplication symbol  $\times$ , and finally a blue rectangle  $V$  with dimensions  $K$  and  $d$ .

PCA is the simplest and by far the most used member of this family. Among all the ways of writing  $X \approx UV$  with  $K$  components, it picks the one minimising the reconstruction error  $\|X - UV\|^2$ , with components orthogonal and ordered by decreasing explained variance.

Two consequences. The components are nested: the first two of a  $K = 32$  decomposition are the same as those of a  $K = 2$  one, which is why PCA is one of the default tools for 2D visualization. And `explained_variance_ratio_` tells you how much of the data you keep; its cumulative sum thus provides a way to choose  $K$ .

Warning: PCA is variance-based, so run it on scaled/normalized features, otherwise the feature with the largest units wins.

Example of reconstructions on MNIST with  $K = 32$



Recall that each image is vectorized, hence each of these images correspond to a row in  $V$ .

2026-08-31

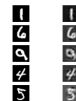
Course 3: Unsupervised Learning

└ Decomposition

└ Principal Components Analysis

Principal Components Analysis

Example of reconstructions on MNIST with  $K = 32$



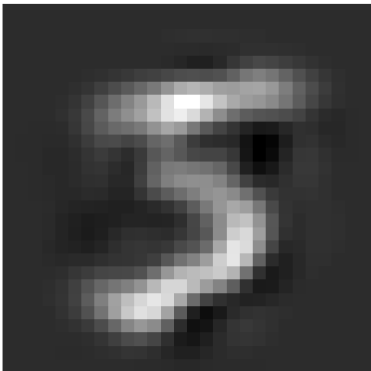
Recall that each image is vectorized, hence each of these images correspond to a row in  $V$ .

32 components instead of 784 pixels, and the digits remain readable: roughly a factor 24, obtained without any label. The slight blur is the visible trace of the discarded variance.

The components themselves are vectors of the same dimension as the images, so they can be displayed as images: they look like “ghost digits”, the first ones capturing the average shape and the main strokes, the last ones the fine details and the noise.

Link with last session: going from  $d = 784$  to  $d = 32$  also mitigates the curse of dimensionality for whatever model comes next.

Detailed example of a reconstruction



2026-08-31

Course 3: Unsupervised Learning

└─ Decomposition

└─ Principal Components Analysis

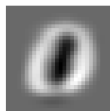
Principal Components Analysis

Detailed example of a reconstruction



Reconstruction of the original vectors from the learnt components. The first overlay shows the results, the following ones unroll the decomposition of a single example by adding one component at a time. Watch how few components are needed before the digit becomes recognisable.



 $= 122.3 \times$ 

Reconstruction of the original vectors from the learnt components. The first overlay shows the results, the following ones unroll the decomposition of a single example by adding one component at a time. Watch how few components are needed before the digit becomes recognisable.

$$\text{Digit 3} = 122.3 \times \text{PC1} - 316.2 \times \text{PC2}$$

2026-08-31

Course 3: Unsupervised Learning

└ Decomposition

└ Principal Components Analysis

Principal Components Analysis

$$\text{Digit 3} = 122.3 \times \text{PC1} - 316.2 \times \text{PC2}$$

Reconstruction of the original vectors from the learnt components. The first overlay shows the results, the following ones unroll the decomposition of a single example by adding one component at a time. Watch how few components are needed before the digit becomes recognisable.



$$\begin{aligned} \text{Digit 3} &= 122.3 \times \text{Component 1} - 316.2 \times \text{Component 2} \\ &\quad - 51.13 \times \text{Component 3} \end{aligned}$$

2026-08-31

Course 3: Unsupervised Learning

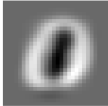
└ Decomposition

└ Principal Components Analysis

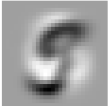
Principal Components Analysis



Reconstruction of the original vectors from the learnt components. The first overlay shows the results, the following ones unroll the decomposition of a single example by adding one component at a time. Watch how few components are needed before the digit becomes recognisable.


$$= 122.3 \times$$

$$- 316.2 \times$$

$$- 51.13 \times$$

$$- 556.9 \times$$


2026-08-31

Course 3: Unsupervised Learning

└ Decomposition

└ Principal Components Analysis

Principal Components Analysis


$$= 122.3 \times$$

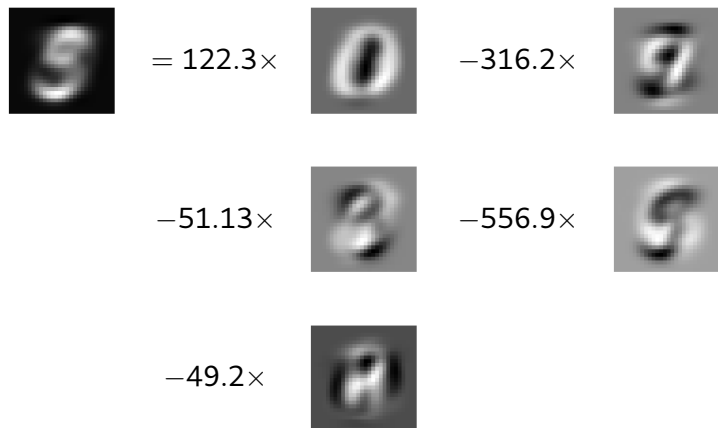
$$- 316.2 \times$$

$$- 51.13 \times$$

$$- 556.9 \times$$


Reconstruction of the original vectors from the learnt components. The first overlay shows the results, the following ones unroll the decomposition of a single example by adding one component at a time. Watch how few components are needed before the digit becomes recognisable.

# Principal Components Analysis



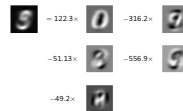
2026-08-31

## Course 3: Unsupervised Learning

### └ Decomposition

### └ Principal Components Analysis

Principal Components Analysis



Reconstruction of the original vectors from the learnt components. The first overlay shows the results, the following ones unroll the decomposition of a single example by adding one component at a time. Watch how few components are needed before the digit becomes recognisable.

# Principal Components Analysis

$$\begin{aligned} \text{5} &= 122.3 \times \text{0} - 316.2 \times \text{9} \\ &- 51.13 \times \text{3} - 556.9 \times \text{5} \\ &- 49.2 \times \text{4} - 217.1 \times \text{6} \dots \end{aligned}$$

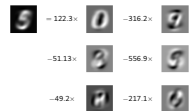
2026-08-31

## Course 3: Unsupervised Learning

### └ Decomposition

### └ Principal Components Analysis

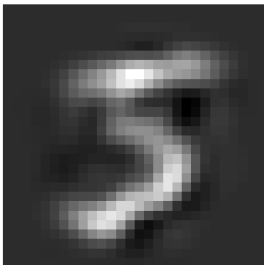
Principal Components Analysis



Reconstruction of the original vectors from the learnt components. The first overlay shows the results, the following ones unroll the decomposition of a single example by adding one component at a time. Watch how few components are needed before the digit becomes recognisable.



Reconstruction with all 32 components:



Reconstruction of the original vectors from the learnt components. The first overlay shows the results, the following ones unroll the decomposition of a single example by adding one component at a time. Watch how few components are needed before the digit becomes recognisable.

# Plan

- 1 Clustering
- 2 Decomposition
- 3 Manifold learning**
- 4 Evaluating unsupervised models
- 5 A note on data cleaning
- 6 Lab session

2026-08-31

## Course 3: Unsupervised Learning

### └─ Manifold learning

#### └─ Plan

Plan

■ Clustering

■ Decomposition

■ **Manifold learning**

■ Evaluating unsupervised models

■ A note on data cleaning

■ Lab session

# Visualizing high dimensional data

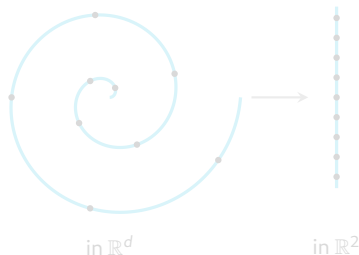
Visualizing high dimensional data helps to **understand its structure**.

But:

- One MNIST digit:  $d = 784$
- 4K image:  $d \approx 25 \cdot 10^6$ .

## Manifold learning

- Hypothesis: high dimensional data lies on (or near) a **lower dimensional** surface.
- Goal: **“unroll”** this surface into 2D/3D, preserving the structure that matters.



The intrinsic dimension can be much smaller than  $d$ !

2026-08-31

## Course 3: Unsupervised Learning

### └ Manifold learning

### └ Visualizing high dimensional data

Visualizing high dimensional data

Visualizing high dimensional data helps to **understand its structure**.

But:

- One MNIST digit:  $d = 784$
- 4K image:  $d \approx 25 \cdot 10^6$ .

#### Manifold learning

- Hypothesis: high dimensional data lies on (or near) a lower dimensional surface.

- Goal: “unroll” this surface into 2D/3D, preserving the structure that matters.

The intrinsic dimension can be much smaller than  $d$ .



Before choosing any model, you will want to plot your data, and your data will not be 2D.

An intuitive example of the manifold hypothesis: photographs of a single object rotating in front of a camera are vectors of millions of pixels, yet the whole set of them is described by one parameter, the rotation angle. The data lives in  $\mathbb{R}^d$  but only fills a 1D curve inside it.

The drawing on the right is the classical “swiss roll”. Two points on facing layers are close in Euclidean distance although they are far apart *along* the surface. That is what these algorithms try to fix, and why PCA (which only knows straight lines) cannot unroll it.

# Visualizing high dimensional data

Visualizing high dimensional data helps to **understand its structure**.

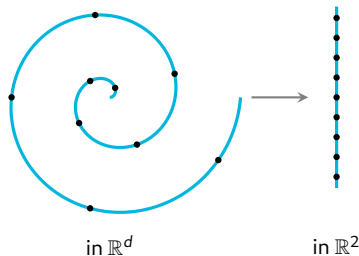
But:

- One MNIST digit:  $d = 784$
- 4K image:  $d \approx 25 \cdot 10^6$ .

## Manifold learning

- Hypothesis: high dimensional data lies on (or near) a **lower dimensional** surface.
- Goal: **"unroll"** this surface into 2D/3D, preserving the structure that matters.

**The intrinsic dimension can be much smaller than  $d$ !**



2026-08-31

## Course 3: Unsupervised Learning

### └ Manifold learning

### └ Visualizing high dimensional data

#### Visualizing high dimensional data

Visualizing high dimensional data helps to **understand its structure**.

But:

- One MNIST digit:  $d = 784$
- 4K image:  $d \approx 25 \cdot 10^6$ .

#### Manifold learning

- Hypothesis: high dimensional data lies on (or near) a **lower dimensional** surface.

- Goal: **"unroll"** this surface into 2D/3D, preserving the structure that matters.

The intrinsic dimension can be much smaller than  $d$ .



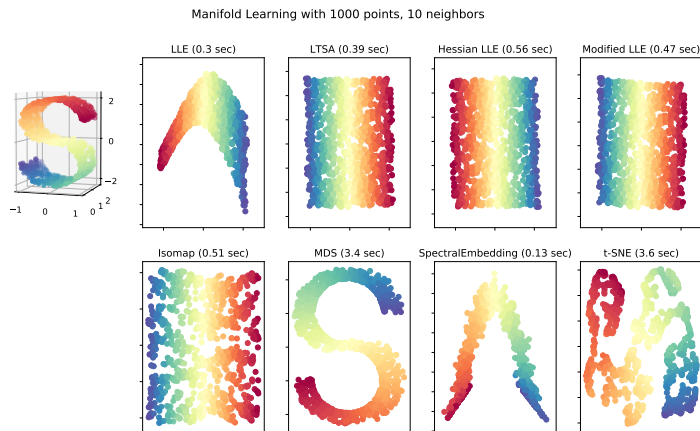
Before choosing any model, you will want to plot your data, and your data will not be 2D.

An intuitive example of the manifold hypothesis: photographs of a single object rotating in front of a camera are vectors of millions of pixels, yet the whole set of them is described by one parameter, the rotation angle. The data lives in  $\mathbb{R}^d$  but only fills a 1D curve inside it.

The drawing on the right is the classical "swiss roll". Two points on facing layers are close in Euclidean distance although they are far apart *along* the surface. That is what these algorithms try to fix, and why PCA (which only knows straight lines) cannot unroll it.



# Manifold learning: a family of methods



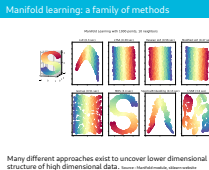
Many different approaches exist to uncover lower dimensional structure of high dimensional data. Source : Manifold module, sklearn website

2026-08-31

## Course 3: Unsupervised Learning

### └ Manifold learning

### └ Manifold learning: a family of methods



We will not detail how each of these methods works. What matters is the range of them: they all take the same input (points in  $\mathbb{R}^d$ ) and produce the same output (points in 2D), but they disagree on what must be preserved (geodesic distances, local neighbourhoods, local linear reconstructions, ...), hence very different pictures of the same dataset.

The original data is the swiss roll shown in the top-left corner of the figure. In practice you may try two or three methods; t-SNE, on the next slide, is the most common default.

# Focus: t-SNE

t-SNE (t-distributed Stochastic Neighbor Embedding): the most popular method to **visualize** high dimensional data.

## How it works (simplified)

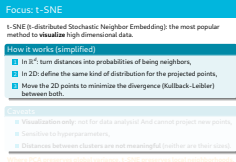
- 1 In  $\mathbb{R}^d$ : turn distances into probabilities of being neighbors,
- 2 In 2D: define the same kind of distribution for the projected points,
- 3 Move the 2D points to minimize the divergence (Kullback-Leibler) between both.

## Caveats

- **Visualization only:** not for data analysis! And cannot project new points,
- Sensitive to hyperparameters,
- **Distances between clusters are not meaningful** (neither are their sizes).

Where PCA preserves global variance, t-SNE preserves local neighborhoods.

2026-08-31 Course 3: Unsupervised Learning  
└─ Manifold learning  
└─ Focus: t-SNE



Stay at the level of intuition here; the details of the Kullback-Leibler divergence are out of scope. The sentence to remember: t-SNE only tries to keep neighbours together, and does not care about what happens far away.

The caveats are what you actually need for the lab and project. The last one is the most frequently violated: a t-SNE plot is not a map, so commenting on the distance between two blobs, or on the size of a blob, is meaningless: both are artefacts of the optimization! A t-SNE plot answers "are there groups?", not "how far apart are they?".

UMAP belongs to the same family and is used in the same way; it is usually faster and preserves a bit more of the global structure.

# Focus: t-SNE

t-SNE (t-distributed Stochastic Neighbor Embedding): the most popular method to **visualize** high dimensional data.

## How it works (simplified)

- 1 In  $\mathbb{R}^d$ : turn distances into probabilities of being neighbors,
- 2 In 2D: define the same kind of distribution for the projected points,
- 3 Move the 2D points to minimize the divergence (Kullback-Leibler) between both.

## Caveats

- **Visualization only:** not for data analysis! And cannot project new points,
- Sensitive to hyperparameters,
- **Distances between clusters are not meaningful** (neither are their sizes).

Where PCA preserves global variance, t-SNE preserves local neighborhoods.

## Course 3: Unsupervised Learning

### └─ Manifold learning

### └─ Focus: t-SNE

2026-08-31

#### Focus: t-SNE

t-SNE (t-distributed Stochastic Neighbor Embedding): the most popular method to **visualize** high dimensional data.

#### How it works (simplified)

- 1 In  $\mathbb{R}^d$ : turn distances into probabilities of being neighbors,
- 2 In 2D: define the same kind of distribution for the projected points,
- 3 Move the 2D points to minimize the divergence (Kullback-Leibler) between both.

#### Caveats

- **Visualization only:** not for data analysis! And cannot project new points,
- Sensitive to hyperparameters,
- **Distances between clusters are not meaningful** (neither are their sizes).

Stay at the level of intuition here; the details of the Kullback-Leibler divergence are out of scope. The sentence to remember: t-SNE only tries to keep neighbours together, and does not care about what happens far away.

The caveats are what you actually need for the lab and project. The last one is the most frequently violated: a t-SNE plot is not a map, so commenting on the distance between two blobs, or on the size of a blob, is meaningless: both are artefacts of the optimization! A t-SNE plot answers "are there groups?", not "how far apart are they?".

UMAP belongs to the same family and is used in the same way; it is usually faster and preserves a bit more of the global structure.

# Focus: t-SNE

t-SNE (t-distributed Stochastic Neighbor Embedding): the most popular method to **visualize** high dimensional data.

## How it works (simplified)

- 1 In  $\mathbb{R}^d$ : turn distances into probabilities of being neighbors,
- 2 In 2D: define the same kind of distribution for the projected points,
- 3 Move the 2D points to minimize the divergence (Kullback-Leibler) between both.

## Caveats

- **Visualization only:** not for data analysis! And cannot project new points,
- Sensitive to hyperparameters,
- **Distances between clusters are not meaningful** (neither are their sizes).

Where PCA preserves global variance, t-SNE preserves local neighborhoods.

## Course 3: Unsupervised Learning

- └ Manifold learning
  - └ Focus: t-SNE

2026-08-31

Focus: t-SNE

t-SNE (t-distributed Stochastic Neighbor Embedding): the most popular method to **visualize** high dimensional data.

How it works (simplified)

- 1 In  $\mathbb{R}^d$ : turn distances into probabilities of being neighbors,
- 2 In 2D: define the same kind of distribution for the projected points,
- 3 Move the 2D points to minimize the divergence (Kullback-Leibler) between both.

Caveats

- **Visualization only:** not for data analysis! And cannot project new points,
- Sensitive to hyperparameters,
- **Distances between clusters are not meaningful** (neither are their sizes).

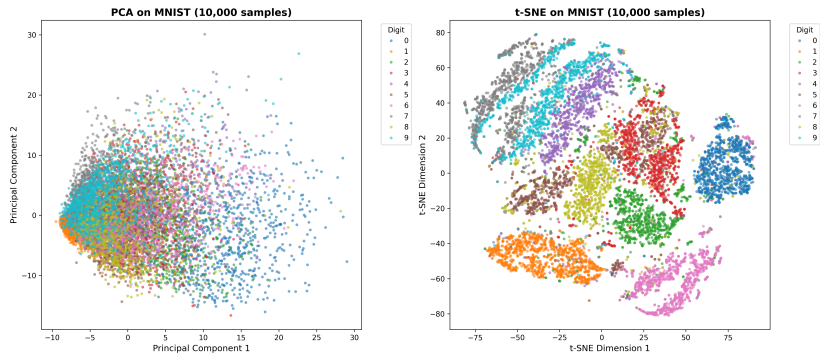
Where PCA preserves global variance, t-SNE preserves local neighborhoods.

Stay at the level of intuition here; the details of the Kullback-Leibler divergence are out of scope. The sentence to remember: t-SNE only tries to keep neighbours together, and does not care about what happens far away.

The caveats are what you actually need for the lab and project. The last one is the most frequently violated: a t-SNE plot is not a map, so commenting on the distance between two blobs, or on the size of a blob, is meaningless: both are artefacts of the optimization! A t-SNE plot answers "are there groups?", not "how far apart are they?".

UMAP belongs to the same family and is used in the same way; it is usually faster and preserves a bit more of the global structure.

# PCA vs t-SNE (illustration on MNIST)



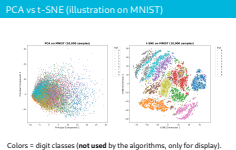
Colors = digit classes (**not used** by the algorithms, only for display).

2026-08-31

## Course 3: Unsupervised Learning

### └ Manifold learning

### └ PCA vs t-SNE (illustration on MNIST)



The same digits projected in 2D by both methods. PCA, being a linear projection, superimposes classes that are not linearly separable in the original space; t-SNE separates them into visible groups.

Two remarks. The colours are labels used only for *display*: both algorithms ran without them. Doing exactly this is the cheapest sanity check of an unsupervised pipeline. And do not over-trust t-SNE: clean groups do not guarantee that a classifier will separate the classes, and PCA remains the tool of choice when the projection must be reused, since it is a real linear transform applicable to new data.

A common practice: run PCA first (down to  $\approx 50$  dimensions) to denoise and speed things up, then t-SNE on the result.

# Plan

- 1 Clustering
- 2 Decomposition
- 3 Manifold learning
- 4 Evaluating unsupervised models**
- 5 A note on data cleaning
- 6 Lab session

2026-08-31

## Course 3: Unsupervised Learning

- └ Evaluating unsupervised models
  - └ Plan

Plan

■ Clustering

■ Decomposition

■ Manifold learning

■ **Evaluating unsupervised models**

■ A note on data cleaning

■ Lab session

# How do we evaluate, without ground truth?

## Evaluation

- Fully unsupervised (internal metrics). Only use  $X$  and the result.  
"Is the structure well-formed?"
- Partly supervised: compare to labels, when they happen to exist.  
"Does the ground-truth structure match what I learned?"
- **Downstream** evaluation: does it help the end task (classification, retrieval, visualization)?

And last but not least: the judgement of a domain expert.

2026-08-31

## Course 3: Unsupervised Learning

### Evaluating unsupervised models

#### How do we evaluate, without ground truth?

#### Evaluation

- Fully unsupervised (internal metrics). Only use  $X$  and the result.  
"Is the structure well-formed?"
- Partly supervised: compare to labels, when they happen to exist.  
"Does the ground-truth structure match what I learned?"
- Downstream evaluation: does it help the end task (classification, retrieval, visualization)?

And last but not least: the judgement of a domain expert.

Last session, evaluation was conceptually simple: there was a right answer, and we counted the mistakes. In unsupervised, we have to assume that this is not possible (even if one can apply unsupervised methods on annotated datasets). When no ground truth is available, you will still have to defend your choices in your project.

All metrics are not always possible. Internal metrics are always available, but they only measure a geometric property that we *decided* was desirable: a dataset with no cluster structure will still hand you a "best"  $K$ . Metrics based on labels are more convincing, but they need labels (in practice they are used on a subset, or on a public dataset, to validate a pipeline before applying it where no label exists). The downstream evaluation is the most honest one in a real project: if the result improves the next model, or helps someone take a decision, it is good.

The take-away: an unsupervised result is never "correct", it is *useful* or not.

# How do we evaluate, without ground truth?

## Evaluation

- Fully unsupervised (internal metrics). Only use  $X$  and the result.  
"Is the structure well-formed?"
- Partly supervised: compare to labels, *when they happen to exist*.  
"Does the ground-truth structure match what I learned?"
- **Downstream** evaluation: does it help the end task (classification, retrieval, visualization)?

And last but not least: the judgement of a domain expert.

2026-08-31

## Course 3: Unsupervised Learning

### Evaluating unsupervised models

#### How do we evaluate, without ground truth?

#### Evaluation

- Fully unsupervised (internal metrics). Only use  $X$  and the result.  
"Is the structure well-formed?"
- Partly supervised: compare to labels, when they happen to exist.  
"Does the ground-truth structure match what I learned?"
- Downstream evaluation: does it help the end task (classification, retrieval, visualization)?

And last but not least: the judgement of a domain expert.

Last session, evaluation was conceptually simple: there was a right answer, and we counted the mistakes. In unsupervised, we have to assume that this is not possible (even if one can apply unsupervised methods on annotated datasets). When no ground truth is available, you will still have to defend your choices in your project.

All metrics are not always possible. Internal metrics are always available, but they only measure a geometric property that we *decided* was desirable: a dataset with no cluster structure will still hand you a "best"  $K$ . Metrics based on labels are more convincing, but they need labels (in practice they are used on a subset, or on a public dataset, to validate a pipeline before applying it where no label exists). The downstream evaluation is the most honest one in a real project: if the result improves the next model, or helps someone take a decision, it is good.

The take-away: an unsupervised result is never "correct", it is *useful* or not.



# How do we evaluate, without ground truth?

## Evaluation

- Fully unsupervised (internal metrics). Only use  $X$  and the result.  
"Is the structure well-formed?"
- Partly supervised: compare to labels, *when* they happen to exist.  
"Does the ground-truth structure match what I learned?"
- **Downstream** evaluation: does it help the end task (classification, retrieval, visualization)?

And last but not least: the judgement of a domain expert.

2026-08-31

## Course 3: Unsupervised Learning

### Evaluating unsupervised models

#### How do we evaluate, without ground truth?

#### Evaluation

- Fully unsupervised (internal metrics). Only use  $X$  and the result.  
"Is the structure well-formed?"
- Partly supervised: compare to labels, *when* they happen to exist.  
"Does the ground-truth structure match what I learned?"
- **Downstream** evaluation: does it help the end task (classification, retrieval, visualization)?

And last but not least: the judgement of a domain expert.

Last session, evaluation was conceptually simple: there was a right answer, and we counted the mistakes. In unsupervised, we have to assume that this is not possible (even if one can apply unsupervised methods on annotated datasets). When no ground truth is available, you will still have to defend your choices in your project.

All metrics are not always possible. Internal metrics are always available, but they only measure a geometric property that we *decided* was desirable: a dataset with no cluster structure will still hand you a "best"  $K$ . Metrics based on labels are more convincing, but they need labels (in practice they are used on a subset, or on a public dataset, to validate a pipeline before applying it where no label exists). The downstream evaluation is the most honest one in a real project: if the result improves the next model, or helps someone take a decision, it is good.

The take-away: an unsupervised result is never "correct", it is *useful* or not.

# How do we evaluate, without ground truth?

## Evaluation

- Fully unsupervised (internal metrics). Only use  $X$  and the result.  
"Is the structure well-formed?"
- Partly supervised: compare to labels, *when* they happen to exist.  
"Does the ground-truth structure match what I learned?"
- **Downstream** evaluation: does it help the end task (classification, retrieval, visualization)?

And last but not least: the judgement of a domain expert.

2026-08-31

## Course 3: Unsupervised Learning

### Evaluating unsupervised models

#### How do we evaluate, without ground truth?

#### Evaluation

- Fully unsupervised (internal metrics). Only use  $X$  and the result.  
"Is the structure well-formed?"
- Partly supervised: compare to labels, *when* they happen to exist.  
"Does the ground-truth structure match what I learned?"
- **Downstream** evaluation: does it help the end task (classification, retrieval, visualization)?

And last but not least: the judgement of a domain expert.

Last session, evaluation was conceptually simple: there was a right answer, and we counted the mistakes. In unsupervised, we have to assume that this is not possible (even if one can apply unsupervised methods on annotated datasets). When no ground truth is available, you will still have to defend your choices in your project.

All metrics are not always possible. Internal metrics are always available, but they only measure a geometric property that we *decided* was desirable: a dataset with no cluster structure will still hand you a "best"  $K$ . Metrics based on labels are more convincing, but they need labels (in practice they are used on a subset, or on a public dataset, to validate a pipeline before applying it where no label exists). The downstream evaluation is the most honest one in a real project: if the result improves the next model, or helps someone take a decision, it is good.

The take-away: an unsupervised result is never "correct", it is *useful* or not.

## Inertia

The error  $E$  of slide 10: sum of squared distances to the centroids.

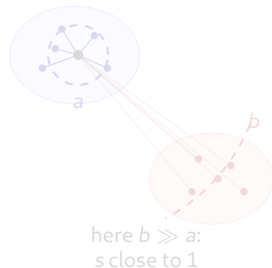
Never increases with  $K$  (0 for  $K = N$ )  $\Rightarrow$  never use it alone.

## Silhouette score

For each sample:  $s = \frac{b - a}{\max(a, b)} \in [-1, 1]$

- $a$ : mean distance to the points of **its own** cluster,
- $b$ : mean distance to the points of the **nearest other** cluster.

$s \approx 1$ : well placed;  $s \approx 0$ : on the border;  
 $s < 0$ : probably in the wrong cluster.



2026-08-31

## Course 3: Unsupervised Learning

### Evaluating unsupervised models

### Fully unsupervised clustering metrics

Fully unsupervised clustering metrics

**Inertia**  
The error  $E$  of slide 10: sum of squared distances to the centroids.  
Never increases with  $K$  (0 for  $K = N$ )  $\Rightarrow$  never use it alone.

**Silhouette score**  
For each sample:  $s = \frac{b - a}{\max(a, b)} \in [-1, 1]$   
 $a$ : mean distance to the points of its own cluster,  
 $b$ : mean distance to the points of the nearest other cluster.  
 $s \approx 1$ : well placed;  $s \approx 0$ : on the border;  
 $s < 0$ : probably in the wrong cluster.



The silhouette is the metric you will use most in the lab, so read the formula on the drawing:  $a$  measures how well the point sits in its own cluster,  $b$  how far it is from the competition, and the score compares the two. The black dot is **one sample among the others**, not a centroid: both  $a$  and  $b$  are averages of distances to individual points, and the two dashed arcs are drawn exactly at those average distances.

Three properties. It is defined *per sample*, so you can plot the whole distribution and see which clusters are badly formed, instead of a single average. It is bounded in  $[-1, 1]$ , so unlike inertia it can be compared across values of  $K$  (which is what makes it usable to choose  $K$ ). And it costs  $O(N^2)$  distances, so on a large dataset compute it on a subsample.

Its bias is worth knowing: like inertia, it rewards compact convex clusters, so it will prefer a K-means solution to a DBSCAN one on non-convex data, even when DBSCAN is visibly right. A metric always encodes a definition of what a good cluster is.

## Inertia

The error  $E$  of slide 10: sum of squared distances to the centroids.

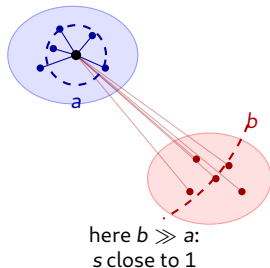
Never increases with  $K$  (0 for  $K = N$ )  $\Rightarrow$  never use it alone.

## Silhouette score

For each sample:  $s = \frac{b - a}{\max(a, b)} \in [-1, 1]$

- $a$ : mean distance to the points of **its own** cluster,
- $b$ : mean distance to the points of the **nearest other** cluster.

$s \approx 1$ : well placed;  $s \approx 0$ : on the border;  
 $s < 0$ : probably in the wrong cluster.



2026-08-31

## Course 3: Unsupervised Learning

### Evaluating unsupervised models

### Fully unsupervised clustering metrics

**Inertia**  
The error  $E$  of slide 10: sum of squared distances to the centroids.  
Never increases with  $K$  (0 for  $K = N$ )  $\Rightarrow$  never use it alone.

**Silhouette score**  
For each sample:  $s = \frac{b - a}{\max(a, b)} \in [-1, 1]$

- $a$ : mean distance to the points of its own cluster,
- $b$ : mean distance to the points of the nearest other cluster.

$s \approx 1$ : well placed;  $s \approx 0$ : on the border;  
 $s < 0$ : probably in the wrong cluster.

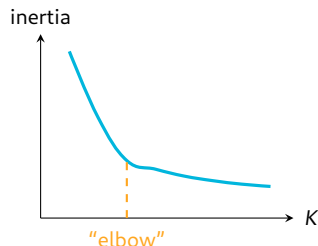


The silhouette is the metric you will use most in the lab, so read the formula on the drawing:  $a$  measures how well the point sits in its own cluster,  $b$  how far it is from the competition, and the score compares the two. The black dot is **one sample among the others**, not a centroid: both  $a$  and  $b$  are averages of distances to individual points, and the two dashed arcs are drawn exactly at those average distances.

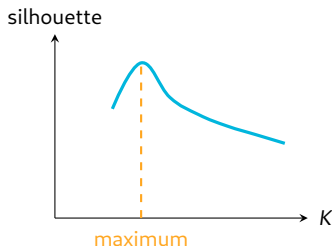
Three properties. It is defined *per sample*, so you can plot the whole distribution and see which clusters are badly formed, instead of a single average. It is bounded in  $[-1, 1]$ , so unlike inertia it can be compared across values of  $K$  (which is what makes it usable to choose  $K$ ). And it costs  $O(N^2)$  distances, so on a large dataset compute it on a subsample.

Its bias is worth knowing: like inertia, it rewards compact convex clusters, so it will prefer a K-means solution to a DBSCAN one on non-convex data, even when DBSCAN is visibly right. A metric always encodes a definition of what a good cluster is.

# Back to choosing $K$ : two heuristics



Elbow: look for the bend



Silhouette: look for the maximum

## In practice

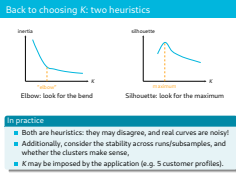
- Both are heuristics: they may disagree, and real curves are noisy!
- Additionally, consider the stability across runs/subsamples, and whether the clusters make sense,
- $K$  may be imposed by the application (e.g. 5 customer profiles).

2026-08-31

## Course 3: Unsupervised Learning

### Evaluating unsupervised models

### Back to choosing $K$ : two heuristics



The recipe for the lab: compute both curves, look at them, decide, and *justify*. These two curves are idealised. On real data the elbow is often invisible, and the silhouette often decreases from  $K = 2$  onwards (a legitimate result that indicates "there is no obvious cluster structure"). You are allowed to conclude that, rather than forcing a value.

The last bullet is what professionals use most: the number of clusters is frequently a constraint of the application (a marketing team handles 5 segments, not 43). The metrics then serve to check that the imposed  $K$  is not absurd. Finally, since K-means depends on its initialization, run it several times and check that the clusters are stable: an unstable clustering is a strong sign that the structure is not there.

# External metrics: when labels do exist

Compare the partition found with a reference partition (the classes). **Never used for training!** Only for validation.

## Comparing with ground-truth labels

- **Rand Index:** agreement between the clustering partitions and the labels. **Invariant to label permutations;**
- **Homogeneity:** each cluster contains only members of a single class,
- **Completeness:** all members of a class are in the same cluster,
- **V-measure:** harmonic mean of both (the "F1" of clustering).



**Warning:** a low score may mean the clustering found *another, valid structure*.

2026-08-31

## Course 3: Unsupervised Learning

### Evaluating unsupervised models

#### External metrics: when labels do exist

External metrics: when labels do exist

Compare the partition found with a reference partition (the classes). **Never used for training!** Only for validation.

Comparing with ground-truth labels

- **Rand Index:** agreement between the clustering partitions and the labels. **Invariant to label permutations;**
- **Homogeneity:** each cluster contains only members of a single class,
- **Completeness:** all members of a class are in the same cluster,
- **V-measure:** harmonic mean of both (the "F1" of clustering).

homogeneous, not complete  
complete, not homogeneous

Rand index ignores permutation, because identifying the meaning of the clusters is not part of the clustering process. Only the grouping matters, not the assignment of clusters with labels (which is why accuracy cannot be used directly here).

The two drawings make homogeneity and completeness concrete. Splitting one class into two perfect halves is homogeneous but not complete; merging two classes into one cluster is complete but not homogeneous. Watch the degenerate cases:  $K = N$  is perfectly homogeneous,  $K = 1$  perfectly complete. Hence the V-measure, which combines both. The similarity with F1-score is not accidental, it kind of is the same idea of balancing over and under-prediction.

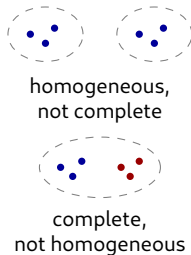
An example for the warning: clustering photographs of people may group them by lighting conditions rather than by identity. Compared to identity labels the score is terrible, although the algorithm found a real and consistent structure. A metric measures agreement with *one particular question*.

# External metrics: when labels do exist

Compare the partition found with a reference partition (the classes). **Never used for training!** Only for validation.

## Comparing with ground-truth labels

- **Rand Index:** agreement between the clustering partitions and the labels. **Invariant to label permutations;**
- **Homogeneity:** each cluster contains only members of a single class,
- **Completeness:** all members of a class are in the same cluster,
- **V-measure:** harmonic mean of both (the "F1" of clustering).



Warning: a low score may mean the clustering found another, valid structure.

2026-08-31

## Course 3: Unsupervised Learning

### Evaluating unsupervised models

#### External metrics: when labels do exist

Rand index ignores permutation, because identifying the meaning of the clusters is not part of the clustering process. Only the grouping matters, not the assignment of clusters with labels (which is why accuracy cannot be used directly here).

The two drawings make homogeneity and completeness concrete. Splitting one class into two perfect halves is homogeneous but not complete; merging two classes into one cluster is complete but not homogeneous. Watch the degenerate cases:  $K = N$  is perfectly homogeneous,  $K = 1$  perfectly complete. Hence the V-measure, which combines both. The similarity with F1-score is not accidental, it kind of is the same idea of balancing over and under-prediction.

An example for the warning: clustering photographs of people may group them by lighting conditions rather than by identity. Compared to identity labels the score is terrible, although the algorithm found a real and consistent structure. A metric measures agreement with *one particular* question.

External metrics: when labels do exist

Compare the partition found with a reference partition (the classes). **Never used for training!** Only for validation.

#### Comparing with ground-truth labels

- **Rand Index:** agreement between the clustering partitions and the labels. **Invariant to label permutations;**
- **Homogeneity:** each cluster contains only members of a single class,
- **Completeness:** all members of a class are in the same cluster,
- **V-measure:** harmonic mean of both (the "F1" of clustering).

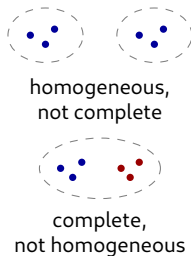


# External metrics: when labels do exist

Compare the partition found with a reference partition (the classes). **Never used for training!** Only for validation.

## Comparing with ground-truth labels

- **Rand Index:** agreement between the clustering partitions and the labels. **Invariant to label permutations;**
- **Homogeneity:** each cluster contains only members of a single class,
- **Completeness:** all members of a class are in the same cluster,
- **V-measure:** harmonic mean of both (the "F1" of clustering).



**Warning: a low score may mean the clustering found another, valid structure.**

2026-08-31

## Course 3: Unsupervised Learning

### Evaluating unsupervised models

#### External metrics: when labels do exist

External metrics: when labels do exist

Compare the partition found with a reference partition (the classes). **Never used for training!** Only for validation.

Comparing with ground-truth labels

- **Rand Index:** agreement between the clustering partitions and the labels. **Invariant to label permutations;**
- **Homogeneity:** each cluster contains only members of a single class,
- **Completeness:** all members of a class are in the same cluster,
- **V-measure:** harmonic mean of both (the "F1" of clustering).

Warning: a low score may mean the clustering found another, valid structure.

Rand index ignores permutation, because identifying the meaning of the clusters is not part of the clustering process. Only the grouping matters, not the assignment of clusters with labels (which is why accuracy cannot be used directly here).

The two drawings make homogeneity and completeness concrete. Splitting one class into two perfect halves is homogeneous but not complete; merging two classes into one cluster is complete but not homogeneous. Watch the degenerate cases:  $K = N$  is perfectly homogeneous,  $K = 1$  perfectly complete. Hence the V-measure, which combines both. The similarity with F1-score is not accidental, it kind of is the same idea of balancing over and under-prediction.

An example for the warning: clustering photographs of people may group them by lighting conditions rather than by identity. Compared to identity labels the score is terrible, although the algorithm found a real and consistent structure. A metric measures agreement with *one particular* question.



## Decomposition (PCA, NMF, ...)

- **Reconstruction error**  $\|X - UV\|^2$ : how much is lost by keeping only  $K$  components,
- **Explained variance ratio**: share of variance carried by each component; its cumulative sum may be used to choose  $K$  ("keep 95%"),
- Careful: reconstruction is perfect for  $K = d$ , but nothing is learned!

## Manifold learning (t-SNE, ...)

- No reconstruction, hence no natural error,
- In practice: **look at the plot**, color it with any metadata you have, and check that your conclusions survive a change of hyperparameters.

Ultimately: does it help the next step of your project?

2026-08-31

## Course 3: Unsupervised Learning

### Evaluating unsupervised models

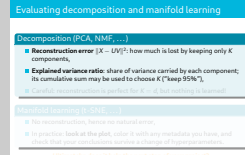
### Evaluating decomposition and manifold learning

Each family gets its own kind of evaluation, and none of them gives a "score out of 10".

For decomposition, the cumulative explained variance is the plot to produce in the lab, and the most defensible way to justify a value of  $K$  in your report. It behaves like the elbow: the curve rises and saturates, and you pick the point where adding components stops paying.

For manifold learning the evaluation is essentially visual, which is exactly why the t-SNE caveats matter. The robustness check is your only real protection: if the groups you see disappear when you change the perplexity, they were an artefact.

The closing sentence is the message of the whole session. In your project, an unsupervised step is justified by what it enables: a cleaner input for the next model, a plot that convinced you of something, a segmentation someone could act on.



## Evaluating decomposition and manifold learning

## Decomposition (PCA, NMF, ...)

- **Reconstruction error**  $\|X - UV\|^2$ : how much is lost by keeping only  $K$  components,
- **Explained variance ratio**: share of variance carried by each component; its cumulative sum may be used to choose  $K$  ("keep 95%"),
- Careful: reconstruction is perfect for  $K = d$ , but nothing is learned!

2026-08-31

## Course 3: Unsupervised Learning

- Evaluating unsupervised models

- Evaluating decomposition and manifold learning

Each family gets its own kind of evaluation, and none of them gives a "score out of 10".

For decomposition, the cumulative explained variance is the plot to produce in the lab, and the most defensible way to justify a value of  $K$  in your report. It behaves like the elbow: the curve rises and saturates, and you pick the point where adding components stops paying.

For manifold learning the evaluation is essentially visual, which is exactly why the t-SNE caveats matter. The robustness check is your only real protection: if the groups you see disappear when you change the perplexity, they were an artefact.

The closing sentence is the message of the whole session. In your project, an unsupervised step is justified by what it enables: a cleaner input for the next model, a plot that convinced you of something, a segmentation someone could act on.

## Evaluating decomposition and manifold learning

- **Decomposition (PCA, NMF, ...)**
  - **Reconstruction error**  $\|X - UV\|^2$ : how much is lost by keeping only  $K$  components,
  - **Explained variance ratio**: share of variance carried by each component; its cumulative sum may be used to choose  $K$  ("keep 95%"),
  - **Careful: reconstruction is perfect for  $K = d$ , but nothing is learned!**
- **Manifold learning (t-SNE, ...)**
  - No reconstruction, hence no natural error,
  - In practice: look at the plot, color it with any metadata you have, and check that your conclusions survive a change of hyperparameters.

## Decomposition (PCA, NMF, ...)

- **Reconstruction error**  $\|X - UV\|^2$ : how much is lost by keeping only  $K$  components,
- **Explained variance ratio**: share of variance carried by each component; its cumulative sum may be used to choose  $K$  ("keep 95%"),
- Careful: reconstruction is perfect for  $K = d$ , but nothing is learned!

## Manifold learning (t-SNE, ...)

- No reconstruction, hence no natural error,
- In practice: **look at the plot**, color it with any metadata you have, and check that your conclusions survive a change of hyperparameters.

Ultimately: does it help the next step of your project?

2026-08-31

## Course 3: Unsupervised Learning

### Evaluating unsupervised models

### Evaluating decomposition and manifold learning

Each family gets its own kind of evaluation, and none of them gives a "score out of 10".

For decomposition, the cumulative explained variance is the plot to produce in the lab, and the most defensible way to justify a value of  $K$  in your report. It behaves like the elbow: the curve rises and saturates, and you pick the point where adding components stops paying.

For manifold learning the evaluation is essentially visual, which is exactly why the t-SNE caveats matter. The robustness check is your only real protection: if the groups you see disappear when you change the perplexity, they were an artefact.

The closing sentence is the message of the whole session. In your project, an unsupervised step is justified by what it enables: a cleaner input for the next model, a plot that convinced you of something, a segmentation someone could act on.

#### Decomposition (PCA, NMF, ...)

- **Reconstruction error**  $\|X - UV\|^2$ : how much is lost by keeping only  $K$  components,
- **Explained variance ratio**: share of variance carried by each component; its cumulative sum may be used to choose  $K$  ("keep 95%"),
- Careful: reconstruction is perfect for  $K = d$ , but nothing is learned!

#### Manifold learning (t-SNE, ...)

- No reconstruction, hence no natural error,
- In practice: **look at the plot**, color it with any metadata you have, and check that your conclusions survive a change of hyperparameters.

## Decomposition (PCA, NMF, ...)

- **Reconstruction error**  $\|X - UV\|^2$ : how much is lost by keeping only  $K$  components,
- **Explained variance ratio**: share of variance carried by each component; its cumulative sum may be used to choose  $K$  ("keep 95%"),
- Careful: reconstruction is perfect for  $K = d$ , but nothing is learned!

## Manifold learning (t-SNE, ...)

- No reconstruction, hence no natural error,
- In practice: **look at the plot**, color it with any metadata you have, and check that your conclusions survive a change of hyperparameters.

Ultimately: does it help the next step of your project?

2026-08-31

## Course 3: Unsupervised Learning

### Evaluating unsupervised models

### Evaluating decomposition and manifold learning

Each family gets its own kind of evaluation, and none of them gives a "score out of 10".

For decomposition, the cumulative explained variance is the plot to produce in the lab, and the most defensible way to justify a value of  $K$  in your report. It behaves like the elbow: the curve rises and saturates, and you pick the point where adding components stops paying.

For manifold learning the evaluation is essentially visual, which is exactly why the t-SNE caveats matter. The robustness check is your only real protection: if the groups you see disappear when you change the perplexity, they were an artefact.

The closing sentence is the message of the whole session. In your project, an unsupervised step is justified by what it enables: a cleaner input for the next model, a plot that convinced you of something, a segmentation someone could act on.

Evaluating decomposition and manifold learning

Decomposition (PCA, NMF, ...)

- **Reconstruction error**  $\|X - UV\|^2$ : how much is lost by keeping only  $K$  components,
- **Explained variance ratio**: share of variance carried by each component; its cumulative sum may be used to choose  $K$  ("keep 95%"),
- Careful: reconstruction is perfect for  $K = d$ , but nothing is learned!

Manifold learning (t-SNE, ...)

- No reconstruction, hence no natural error,
- In practice: **look at the plot**, color it with any metadata you have, and check that your conclusions survive a change of hyperparameters.

Ultimately: does it help the next step of your project?

# Plan

- 1 Clustering
- 2 Decomposition
- 3 Manifold learning
- 4 Evaluating unsupervised models
- 5 A note on data cleaning**
- 6 Lab session

2026-08-31

## Course 3: Unsupervised Learning

└ A note on data cleaning

└ Plan

Plan

■ Clustering

■ Decomposition

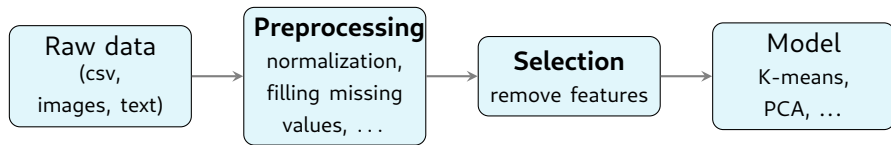
■ Manifold learning

■ Evaluating unsupervised models

■ **A note on data cleaning**

■ Lab session

# Features in standard Machine Learning pipelines



## Why it matters here

- Every method seen today relies on **distances** or **variance**,
- Both are **not invariant** to the units and the scale of the features,

*"Garbage in, garbage out"*

Data preparation may matter more than the model itself!

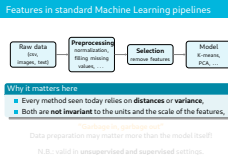
N.B.: valid in **unsupervised** and **supervised** settings.

2026-08-31

## Course 3: Unsupervised Learning

### └ A note on data cleaning

### └ Features in standard Machine Learning pipelines

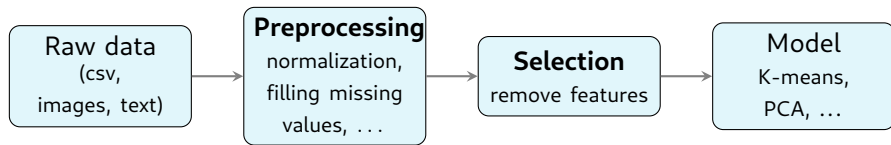


The "boring but decisive" part of the pipeline (at least for standard Machine Learning).

Here is why it closes the session, after the three families of methods and their evaluation: K-means compares distances, PCA maximises variance, t-SNE compares neighbourhoods. All three change completely if one feature is expressed in euros and another in thousands of euros. Unlike some supervised methods (e.g., decision tree), these methods cannot compensate for a bad scaling.

In scikit-learn everything here is a `fit/transform` object, and the natural way to chain them is a `Pipeline`, which also guarantees that the transformations are fitted on the training data only.

# Features in standard Machine Learning pipelines



## Why it matters here

- Every method seen today relies on **distances** or **variance**,
- Both are **not invariant** to the units and the scale of the features,

**"Garbage in, garbage out"**

Data preparation may matter more than the model itself!

N.B.: valid in **unsupervised** and **supervised** settings.

2026-08-31

## Course 3: Unsupervised Learning

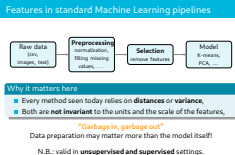
### A note on data cleaning

### Features in standard Machine Learning pipelines

The "boring but decisive" part of the pipeline (at least for standard Machine Learning).

Here is why it closes the session, after the three families of methods and their evaluation: K-means compares distances, PCA maximises variance, t-SNE compares neighbourhoods. All three change completely if one feature is expressed in euros and another in thousands of euros. Unlike some supervised methods (e.g., decision tree), these methods cannot compensate for a bad scaling.

In scikit-learn everything here is a `fit/transform` object, and the natural way to chain them is a `Pipeline`, which also guarantees that the transformations are fitted on the training data only.



**Objective:** change the values of the features, without removing any.

## Classical preprocessing tools

- **Encoding:** turn categories into numbers (OneHotEncoder), discretize continuous values,
- **Scaling / standardization:** same mean and variance for all features (StandardScaler), or same range (MinMaxScaler),
- **Normalization:** put each *sample* on the unit sphere (Normalizer),
- **Power / quantile transforms:** make a skewed feature more Gaussian,
- **Manual feature engineering:** build the feature the model cannot guess (ratios, dates → day of week).

<https://scikit-learn.org/stable/modules/preprocessing.html>

2026-08-31 Course 3: Unsupervised Learning  
└ A note on data cleaning  
└ Feature preprocessing

Examples of feature preprocessing methods.

Do not mix up the scaling and normalization: scaling acts *column-wise* (per feature, over the dataset), normalization acts *row-wise* (per sample, dividing by its norm). Normalization is what you want when only the direction of the vector matters, e.g. word counts in a document, where long documents should not end up far from everything.

Feature preprocessing

Objective: change the values of the features, without removing any.

Classical preprocessing tools

- **Encoding:** turn categories into numbers (to text to doc or  $x$ ), discretize continuous values,
- **Scaling / standardization:** same mean and variance for all features (to standardize  $x$ ), or same range (to scale  $x$ ),
- **Normalization:** put each sample on the unit sphere (to real  $x$ ),
- **Power / quantile transforms:** make a skewed feature more Gaussian,
- **Manual feature engineering:** build the feature the model cannot guess (ratios, dates → day of week).



**Objective:** remove useless features.

Useful for: fighting the curse of dimensionality, speeding up computations, keeping results interpretable, ...

## Selecting the features

- Remove features with low variance (`VarianceThreshold`): a constant column carries no information,
- Remove redundant features (strongly correlated with another one).

Note: PCA is *not* feature selection! It builds new features as combinations of all of them.

[https://scikit-learn.org/stable/modules/feature\\_selection.html](https://scikit-learn.org/stable/modules/feature_selection.html)

2026-08-31

## Course 3: Unsupervised Learning

└ A note on data cleaning

└ Feature selection

Selection differs from preprocessing in one respect: it *destroys* information, so it must be justified. The good reasons are on the slide: fewer dimensions means less overfitting for the next model, faster computations, and a result you can explain to a domain expert ("these 5 measurements are enough").

Without labels you can only remove what is constant or redundant. If you do have labels, other criteria exist (`SelectKBest`, or the feature importances of a model), but consist in supervision, even if weaker than full supervision (like using a supervised model end-to-end). Selection must be considered as part of the model.

**Objective:** remove useless features.  
Useful for: fighting the curse of dimensionality, speeding up computations, keeping results interpretable, ...

### Selecting the features

- Remove features with low variance (`VarianceThreshold`): a constant column carries no information,
- Remove redundant features (strongly correlated with another one).

Note: PCA is not feature selection! It builds new features as combinations of all of them.

15/02/2026 10:10:10 AM - 10/10/2026 10:10:10 AM - 10/10/2026 10:10:10 AM

# Plan

- 1 Clustering
- 2 Decomposition
- 3 Manifold learning
- 4 Evaluating unsupervised models
- 5 A note on data cleaning
- 6 Lab session

2026-08-31

## Course 3: Unsupervised Learning

└ Lab session

└ Plan

Plan

■ Clustering

■ Decomposition

■ Manifold learning

■ Evaluating unsupervised models

■ A note on data cleaning

■ Lab session

# Lab Session 3: Unsupervised Learning

## Part 1 — Toy datasets, with `sklearn`

- **Generate toy datasets** (blobs, moons, ...),
- Preprocess/select features, then cluster with `KMeans`,
- Decompose with `PCA`, visualise with `PCA` and `TSNE`,
- Find **where K-means breaks**, and try a different clustering method fitting the shape of the data.

Then, do the same on a **real dataset**, and explore a different clustering approach.

You may also explore `DBSCAN`, agglomerative clustering, Gaussian Mixture Models, spectral clustering; you may also try `UMAP` for visualisation.

Link to the lab: <https://mee-labs.gitlab-pages.imt-atlantique.fr/intro2ai/lab3/>

## Part 2 — Your own project data

Apply the very same pipeline to the dataset of **your project**.

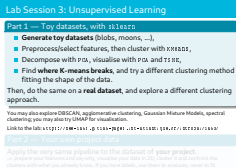
⇒ prepare your features and say why, visualise your data in 2D, cluster it and confront the clusters with what you already know. If you have labels, use them to *evaluate*, never to fit.

2026-08-31

## Course 3: Unsupervised Learning

└ Lab session

└ Lab Session 3: Unsupervised Learning



Part 1 is the standard application of the principles of the course. There is no "best" unsupervised algorithm, only algorithms whose assumptions match (or not) the shape of your data. For instance, two intertwined moons are trivial for a density-based method and impossible for K-means, whatever you tune. Build the datasets that break the model on purpose, it is the fastest way to understand it!

Part 2 will help you better understand your data and project. You should keep your notebook and your conclusions, it may be useful for your final report!

The most useful outcome for most projects is the 2D visualization: it is the fastest way to notice that your data is duplicated, unbalanced, dominated by a single feature, or already separated. Colour your projection with everything you have (a category, a date, a source) even when it is not your target.

This lab is exploratory: the quality of the exploration and of the justification matters more than the beauty of the final plot. A negative result that you correctly diagnose is worth more than a lucky one.

# Lab Session 3: Unsupervised Learning

## Part 1 — Toy datasets, with `sklearn`

- **Generate toy datasets** (blobs, moons, ...),
- Preprocess/select features, then cluster with `KMeans`,
- Decompose with `PCA`, visualise with `PCA` and `TSNE`,
- Find **where K-means breaks**, and try a different clustering method fitting the shape of the data.

Then, do the same on a **real dataset**, and explore a different clustering approach.

You may also explore DBSCAN, agglomerative clustering, Gaussian Mixture Models, spectral clustering; you may also try UMAP for visualisation.

Link to the lab: <https://mee-labs.gitlab-pages.imt-atlantique.fr/intro2ai/lab3/>

## Part 2 — Your own project data

Apply the very same pipeline to the dataset of **your project**.

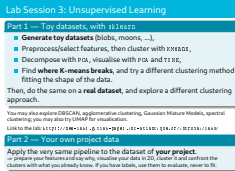
⇒ prepare your features and say why, visualise your data in 2D, cluster it and confront the clusters with what you already know. If you have labels, use them to *evaluate*, never to fit.

2026-08-31

## Course 3: Unsupervised Learning

└ Lab session

└ Lab Session 3: Unsupervised Learning



Part 1 is the standard application of the principles of the course. There is no “best” unsupervised algorithm, only algorithms whose assumptions match (or not) the shape of your data. For instance, two intertwined moons are trivial for a density-based method and impossible for K-means, whatever you tune. Build the datasets that break the model on purpose, it is the fastest way to understand it!

Part 2 will help you better understand your data and project. You should keep your notebook and your conclusions, it may be useful for your final report!

The most useful outcome for most projects is the 2D visualization: it is the fastest way to notice that your data is duplicated, unbalanced, dominated by a single feature, or already separated. Colour your projection with everything you have (a category, a date, a source) even when it is not your target.

This lab is exploratory: the quality of the exploration and of the justification matters more than the beauty of the final plot. A negative result that you correctly diagnose is worth more than a lucky one.